

UNIVERSIDAD NACIONAL DE SAN AGUSTÍN

ESCUELA PROFESIONAL DE CIENCIA DE LA COMPUTACIÓN

Multiplicación de Matrices y Trapecio

Informe final reproducible de optimización MPI

Ethernet, Wi-Fi, difusión jerárquica y 243 registros documentados

Versión revisada con auditoría de la evidencia y análisis de variabilidad

Integrantes:

Paredes Malaga José Carlos
Jheeremy Manuel Alvarez Astete
Jose Alejandro Machaca Muñiz
David Tamo Turpo
Wilson Ramos

Curso:

Computación Paralela y Distribuida

Profesor:

Alvaro Henry Mamani Aliaga

Repositorio: <https://github.com/dtamotu/cluster-mpi-96-optimizaciones>

Arequipa, Perú – 25 de septiembre de 2026

Índice

1. Resumen ejecutivo	1
1.1. Respuestas breves	1
1.2. Qué cambió respecto de la imagen inicial	1
1.3. Qué añade esta versión revisada	2
1.4. Alcance y reproducibilidad	2
2. Introducción	2
2.1. Qué datos usa este informe	3
2.2. Cómo leer las cifras	3
3. Nuestro clúster	3
3.1. Cuántos procesos se colocan en cada nodo	4
4. Cómo funcionan los programas	4
4.1. Trapecio: repartir un intervalo y sumar resultados	4
4.2. Matrices: filas entre procesos y bloques dentro de cada proceso	5
5. Diseño con la metodología de Foster	5
5.1. Qué se varía y qué se mantiene	6
6. Cómo medimos y comparamos	6
6.1. El tiempo que queda en el CSV	6
6.2. Speedup, eficiencia y comparación entre redes	6
6.3. Por qué las fases no se suman	7
6.4. Comprobaciones de consistencia	7
7. Qué mejoró al trabajar por bloques	7
7.1. Evidencia histórica en una sola computadora	7
7.2. Qué significa la estimación de 232 GB	7
8. Primera batería: resultados del trapecio	8
8.1. Problema pequeño: $n = 10^8$	8
8.2. Por qué la Wi-Fi casi no importa con 24 procesos	9
8.3. Problema grande: $n = 10^{11}$	9
8.4. Por qué la eficiencia no llega al 100 %	9
9. Primera batería: multiplicación de matrices	10
9.1. Matriz 1024×1024	10
9.2. Matriz 2048×2048	10
9.3. Matriz 3072×3072	10
9.4. Más procesos pueden aumentar el tiempo	11
9.5. Dónde se va el tiempo	11
9.6. Verificación de las matrices calculadas	12

10. Segunda batería: resultados ampliados del 25 de septiembre	12
10.1. Trapecio: el trabajo largo sigue escalando	12
10.2. Matrices: un nodo completo sigue siendo el mejor punto	13
10.3. Qué miden los MB nuevos	14
10.4. Exploración de núcleos P y E	15
11. La madrugada: 906 segundos y el costo de difundir B	15
11.1. Caso central: el comando y el registro de las 00:58	15
11.2. Segunda serie: medir tiempo y tráfico	16
11.3. Los MB medidos durante la madrugada	16
11.4. Comparación histórica del mediodía	17
12. Qué cambió en las instrucciones de lanzamiento	18
12.1. Comando histórico	18
12.2. Comando actual	18
12.3. Qué significa cada opción	19
12.4. Memoria compartida no demuestra difusión jerárquica	19
13. Cómo interpretar los MB transmitidos	19
13.1. Medición real y modelo de tráfico	19
13.2. Cuánto pesa una matriz	20
14. Campaña final de optimización	20
14.1. Protocolo y control de calidad	20
14.2. Resumen de las comparaciones	21
14.3. Difusión jerárquica: la mejora demostrada	22
14.4. Partición 2D y tamaño del problema	23
14.5. Reparto asimétrico del trapecio	23
14.6. Tabla completa de la campaña	24
14.7. Resultados que no deben sobreinterpretarse	26
15. Una corrida frente a repeticiones	26
15.1. Qué muestra la comparación	26
15.2. Consecuencias para la lectura del informe	27
16. Auditoría de la evidencia	27
16.1. Lo que queda verificado	27
16.2. Problemas hallados y cómo se tratan	28
17. Conclusiones y límites de lo que podemos defender	29
17.1. Conclusiones	29
17.2. Qué cambió respecto del informe anterior	30
17.3. Qué haría falta para confirmar las causas pendientes	30
A. Configuración, compilación y comprobación	31

A.1. Comprobar el software y los nodos	31
A.2. Compilar los dos programas	31
A.3. Copiar y verificar los binarios	31
B. Comandos de lanzamiento y medición	32
B.1. Una ejecución, paso a paso	32
B.2. Batería completa y CSV	32
B.3. Qué hace el script de medición	32
B.4. Regenerar gráficos de las corridas actuales	32
B.5. Reconstruir este informe	33
B.6. Lectura de un resultado real	33
C. Cómo registrar los MB en una próxima ejecución	33
C.1. Aislar el efecto de una opción	34
D. Scripts actuales conservados	34
D.1. Lanzador Wi-Fi	34
D.2. Medición Bash y generación de gráficos	35
E. Archivos de respaldo y fuentes	36

1. Resumen ejecutivo

Este documento reúne y vuelve a interpretar toda la evidencia disponible del proyecto de multiplicación de matrices y regla del trapecio en un clúster de cuatro nodos con 96 núcleos físicos. Se mantienen separados los experimentos históricos, la batería ampliada y la campaña de optimización para que cada comparación conserve su protocolo y su alcance.

Fuente	Registros	Qué permite afirmar
Primera batería	50	Comparación inicial entre Ethernet y Wi-Fi con $P = 1, 24, 48, 72, 96$; una corrida por configuración.
Batería ampliada	75	Repite tamaños y añade $N = 512, 4096, 6144$; conserva TX+RX del servidor y comprobaciones de consistencia.
Optimización controlada	118	Medianas y rangos de 3 o 5 repeticiones; compara difusión jerárquica, SUMMA 2D, algoritmos tuned, tamaños mayores y reparto asimétrico.
Total documentado	243	Registros de ejecución conservados en CSV, logs, metadatos y sellos SHA-256.

1.1. Respuestas breves

Pregunta	Respuesta sustentada por los datos	Evidencia
¿Conviene usar los cuatro nodos para matrices?	No en ningún tamaño medido ($N \leq 7168$). El mejor tiempo siempre fue un nodo con 24 procesos; con 96 procesos, 1D global tardó entre 4.57 y 9.65 veces más por Ethernet.	Secc. 14, tabla 1
¿Qué optimización funcionó?	Difundir B una vez por nodo. Con $N = 3072$ y $P = 96$, la mediana bajó de 6.260 a 3.217 s por Ethernet y de 101.082 a 51.479 s por Wi-Fi; el TX agregado bajó a menos de la mitad.	Secc. 14
¿Hace falta programarla a mano?	En la difusión aislada, pedir a Open MPI el algoritmo <code>knomial</code> produjo un TX similar y una mediana 3 % menor que la versión jerárquica escrita en C. Falta medirlo en el programa completo.	Tabla 1
¿Mejoró SUMMA 2D?	No. Con $P = 64$ tardó 1.75 veces más que 1D por Ethernet y movió casi el doble de datos.	Tabla 1
¿Conviene distribuir el trapecio?	Sí, con carga grande: $n = 10^{11}$ alcanza un speedup de 68.56 con 96 procesos por Ethernet. Con $n = 10^8$ por Wi-Fi, 96 procesos son más lentos que uno.	Secc. 8.3
¿Ayuda el reparto asimétrico P/E?	No de forma concluyente. Con 24 procesos y $n = 10^{11}$, la mediana mejora 1.05 veces pero los rangos se solapan; con 96 procesos es 3 % más lento.	Tabla 2
¿Son estables las corridas únicas?	En las configuraciones de matrices comparables con 96 procesos, las corridas únicas quedaron a menos de un 2 % de las medianas posteriores. Con 24 procesos la campaña fue 19–37 % más lenta, sobre todo en el cálculo, y se compiló sin <code>-march=native</code> .	Secc. 15

1.2. Qué cambió respecto de la imagen inicial

La ejecución de madrugada con ocho procesos repartidos entre cuatro nodos (dos por nodo), $N = 3072$ y Wi-Fi tardó 906.351 s y registró 889.744 s en el máximo de `MPI_Bcast(B)`. Ese

caso sigue siendo un antecedente válido, pero no se mezcla con las campañas nuevas: usó otro ejecutable, otro kernel y otra instrumentación. La campaña controlada repite la pregunta con afinidad fija, kernel por bloques, comprobaciones de huellas y una separación explícita entre 1D global y 1D jerárquica. Por eso ahora se puede atribuir la mejora a una hipótesis comprobada en esta topología, sin afirmar que el comando por sí solo revele el algoritmo interno de Open MPI.

1.3. Qué añade esta versión revisada

- Una **auditoría completa** de la evidencia (sección 16): las 243 filas de resultados coinciden con sus logs, las huellas numéricas son consistentes y los sellos SHA-256 están intactos. También documenta los problemas de trazabilidad hallados en archivos auxiliares y borradores.
- Una comparación entre las **corridas únicas y las medianas repetidas** (sección 15), que acota cuánto se puede confiar en cada cifra de las baterías.
- La advertencia de que la campaña se compiló **sin -march=native -funroll-loops**, a diferencia de las baterías: sus comparaciones internas son válidas, pero sus tiempos absolutos de cálculo no son directamente comparables con los de las baterías.
- Una tabla resumen de la campaña con cocientes, solapamiento de rangos y el resultado **knomial**, más la tabla de repeticiones del trapecio con el desbalance leído de cada log.
- Conclusiones unificadas al final, después de toda la evidencia, y figuras de la campaña con valores anotados y en formato vectorial.

1.4. Alcance y reproducibilidad

Los tiempos provienen de `MPI_Wtime`; los contadores de red se describen junto a su alcance y no se presentan como bytes útiles exactos de una colectiva. Cada corrida nueva conserva su comando, entorno, log y resultado en `resultados_optimizacion/`. Los scripts de esta carpeta regeneran tablas, figuras, auditorías y este PDF sin depender de los nodos remotos; leen la evidencia del repositorio original sin modificarla.

2. Introducción

Este laboratorio estudia cuándo conviene repartir un cálculo entre varias computadoras. Usamos un clúster de **cuatro nodos y 96 núcleos físicos**, con dos problemas: integrar una función mediante la regla del trapecio y multiplicar matrices. Comparamos una ejecución con un proceso, una computadora con 24 procesos y hasta cuatro computadoras con 96 procesos. También comparamos la comunicación por Ethernet y por Wi-Fi.

La pregunta central es: **¿cuánto tiempo ahorramos al repartir el cálculo y cuánto perdemos al comunicar los datos?** Los dos problemas permiten observar situaciones diferentes. Trapecio realiza mucho cálculo y comunica un resultado pequeño. Las matrices necesitan distribuir y reunir grandes cantidades de datos.

Lo que encontramos. Las dos baterías coinciden en la conclusión principal: el trapecio grande aprovecha los 96 procesos, mientras que para todas las matrices medidas el mejor tiempo se obtiene con **24 procesos en una computadora**. La segunda batería amplía los tamaños hasta $N = 6144$ por Ethernet y mide también los contadores TX+RX del servidor.

El caso que motivó el estudio: 15 minutos por Wi-Fi. A las 00:58:05 del 24 de septiembre, multiplicar matrices de 3072×3072 con **8 procesos distribuidos entre cuatro nodos, dos en cada uno** (servidor, workers1, workers2 y workers4), tardó **906.351 s**. El log registró un máximo de **889.744 s en MPI_Bcast(B)** y solo **1.735 s de cálculo**. El cociente $889.744/906.351 = 98.17\%$ muestra dónde se concentró el tiempo registrado; los máximos por fase pueden corresponder a rangos distintos y no se suman. En esta ejecución no se midieron MB transmitidos.

2.1. Qué datos usa este informe

El informe distingue dos baterías, sin mezclar sus tiempos para calcular promedios. La primera, del 24 de septiembre, contiene **25 configuraciones por red**, con una ejecución de cada una: diez de trapecio y quince de matrices. Los resultados originales se conservaron en:

- Ethernet: `experimentos_96/resultados/20260924_202548_RnIhhM/`.
- Wi-Fi: `experimentos_96/resultados_wifi/20260924_203957_VSNYOP/`.

Las rutas parten de `/home/alumno16/`. Las tablas de esa primera batería se generaron leyendo los CSV y cotejándolos con las líneas `RESULT_TRAP` y `RESULT_2D` de los 50 logs.

La segunda batería, del 25 de septiembre, está en `experimentos_96/bateria_expandida/20260925_003457/`: **75 ejecuciones**, con 40 casos Ethernet y 35 Wi-Fi. El informe la presenta en una sección propia y conserva sus 75 logs, CSV, JSON y el borrador `INFORME_ANALISIS_EXPANDIDO_96.md` como fuentes. El borrador se revisó contra los registros antes de incorporar sus afirmaciones; la sección 16 resume qué partes resistieron esa revisión.

El código, los datos y la fuente de este informe están versionados en el repositorio privado derivado <https://github.com/dtamotu/cluster-mpi-96-optimizaciones>.

Los registros de madrugada y mediodía se usan como **antecedentes identificados por hora, versión, número de procesos y repeticiones**. No se mezclan con las dos baterías para calcular promedios. En particular, la prueba de 906 s y la prueba que transmitió 1112 MB fueron ejecuciones diferentes.

2.2. Cómo leer las cifras

Todos los tiempos son segundos medidos dentro del programa con `MPI_Wtime`. Se usa punto decimal. Los tamaños de tráfico están en **MB decimales**, donde $1 \text{ MB} = 10^6$ bytes. Los gráficos de las dos baterías usan escala lineal y muestran los valores; algunas figuras separan las redes con escalas verticales diferentes para hacer visibles sus resultados. Las figuras de la campaña de optimización usan escala logarítmica, indicada en su eje, porque comparan tiempos que difieren en dos órdenes de magnitud. Cuando hay repeticiones se informa la **mediana** y el rango mínimo-máximo, no la media ni un intervalo de confianza.

3. Nuestro clúster

Se conservan los cuatro equipos del trabajo anterior. El servidor también calcula: no está reservado exclusivamente a coordinar.

Nodo	Usuario	Ethernet	Wi-Fi
servidor	alumno16	10.7.50.202	10.7.134.117
workers1	alumno22	10.7.50.203	10.7.134.58
workers2	alumno21	10.7.50.201	10.7.134.59
workers4	alumno6	10.7.50.204	10.7.134.51

Componente	Configuración documentada
CPU por nodo	Intel Core Ultra 9 285; 24 núcleos físicos sin Hyper-Threading; 8 P y 16 E.
Total	$4 \times 24 = 96$ núcleos físicos.
RAM por nodo	32 GB DDR5-5600 en un módulo. Ancho de banda teórico del canal de 64 bits: 44.8 GB/s.
Caché	L2 por núcleo P y compartida por grupos de núcleos E; L3 compartida de 36 MB.
Software	Ubuntu 24.04 y Open MPI 4.1.6, según el inventario del proyecto.
Ethernet	Enlace Gigabit; interfaz enp128s31f6 , subred 10.7.50.0/24. Switch compartido de la universidad.
Wi-Fi	Red institucional compartida; interfaz wlp129s0f0 , subred 10.7.134.0/23.

El equipo **workers3** se excluyó por incompatibilidad del entorno MPI/PMIx. Los inventarios históricos describen las máquinas, pero no prueban que frecuencia, señal o carga de fondo fueran iguales durante todas las ejecuciones. Por ejemplo, el inventario de las 06:46 documentó un perfil de energía diferente en **workers1**.

3.1. Cuántos procesos se colocan en cada nodo

P total	servidor	workers1	workers2	workers4	Nodos
1	1	0	0	0	1
24	24	0	0	0	1
48	24	24	0	0	2
72	24	24	24	0	3
96	24	24	24	24	4

Cada proceso MPI ejecuta su cálculo local sin OpenMP. Por eso hablamos de **procesos**, no de un número configurable de hilos. El lanzamiento actual fija cada proceso a un núcleo. $P = 1$ es una referencia con el mismo programa MPI; no es una implementación secuencial independiente. $P = 24$ permite observar el paralelismo dentro de un nodo, sin transferencias de las matrices entre computadoras.

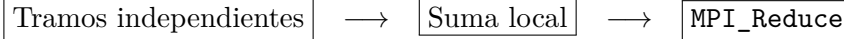
4. Cómo funcionan los programas

4.1. Trapecio: repartir un intervalo y sumar resultados

Se aproxima

$$I = \int_0^1 \frac{4}{1+x^2} dx = \pi$$

con n trapecios de ancho $h = 1/n$. Cada proceso calcula un tramo contiguo. Cuando n no es múltiplo de P , los primeros $n \bmod P$ procesos reciben un trapecio adicional. Al final, **MPI_Reduce** suma los resultados parciales en el proceso 0.



Durante el intervalo medido hay una reducción del resultado: **un double por proceso**, además de la coordinación propia de MPI. Después del cronometraje se ejecutan otras cuatro reducciones para resumir los tiempos. No se debe confundir el tamaño del dato matemático con todos los bytes transmitidos por MPI, TCP y SSH.

4.2. Matrices: filas entre procesos y bloques dentro de cada proceso

Se calcula $C = AB$, con matrices cuadradas de dobles. Ambas baterías ejecutan:

```
1 /var/tmp/mpi_bloques/mpi_matrix_2d N 1d tiled 64
```

El nombre del ejecutable conserva **2d**, pero el argumento **1d** selecciona **franjas de filas**. Los pasos son:

1. **MPI_Scatterv**: el proceso 0 reparte las filas de A .
2. **MPI_Bcast**: todos los procesos obtienen una copia completa de B .
3. Cada proceso calcula sus filas de C usando el kernel **tiled**.
4. **MPI_Gatherv**: el proceso 0 reúne las filas de C .

Bcast distribuye el mismo dato a todos; **Gatherv** reúne partes de distintos tamaños en un destino. Las variantes con **v** permiten repartir filas aunque N no sea múltiplo de P . Con $N = 3072$ y $P = 96$, cada proceso recibe exactamente 32 filas.

Bloques de caché y partición 2D son cambios diferentes. El kernel **tiled 64** reorganiza el cálculo local en teselas para reutilizar datos. SUMMA distribuye bloques de las matrices en una malla de procesos; esta implementación exige $P = q^2$ y N divisible por q . Como 96 no es cuadrado perfecto, ambas baterías usan 1D. Esto es una restricción del código disponible, no una imposibilidad general de usar una malla rectangular con 96 procesos.

5. Diseño con la metodología de Foster

La metodología de Foster organiza el diseño paralelo en partición, comunicación, agrupación y asignación [1]. Aplicada a estos programas:

Etapa	Trapezio	Multipliación 1D
Partición	Dividir los n trapecios en unidades de trabajo.	Identificar filas de C que pueden calcularse de forma independiente si se dispone de A y B .
Comunicación	Sumar las integrales parciales con MPI_Reduce .	Repartir A , difundir B y reunir C .
Agrupación	Agrupar trapecios contiguos en un tramo por proceso; calcular antes de comunicar.	Agrupar filas por proceso. Dentro del proceso, agrupar operaciones en teselas de 64×64 para reutilizar caché.
Asignación	Colocar un proceso por núcleo y llenar nodos de 24 núcleos.	Usar la misma asignación; las comunicaciones locales pueden usar memoria compartida y las remotas la red.

5.1. Qué se varía y qué se mantiene

Para cada tamaño se mantiene fijo el problema y se cambia P : es una prueba de **escalado fuerte**. No se aumenta el tamaño proporcionalmente a los procesos.

Experimento	Tamaños	Procesos
Trapezio	$10^8, 10^{11}$	1, 24, 48, 72, 96
Matrices 1D-tiled	1024, 2048, 3072	1, 24, 48, 72, 96

En la primera batería se ejecutaron las mismas 25 configuraciones en las dos redes. Se completó primero Ethernet y después Wi-Fi, sin alternancia ni aleatorización. La segunda amplió los tamaños y ejecutó 40 casos por Ethernet y 35 por Wi-Fi. Se realizó una sola ejecución por configuración en ambas.

Los resultados de ambas baterías describen **lo observado en esas ejecuciones**. Una medición permite calcular tiempo, speedup y eficiencia, pero no permite estimar dispersión, intervalos de confianza o estabilidad. Las diferencias pequeñas entre redes deben interpretarse con especial cuidado.

La selección de tamaños ofrece dos contrastes: trapezio pequeño hace visible el costo de coordinación; trapezio grande lo amortiza. En matrices, añadir procesos reduce el cálculo local, pero no elimina la obligación de mover A , B y C . Pasar de 24 a 48 procesos cambia a la vez el número de núcleos y el número de nodos: ahí aparece la comunicación entre computadoras.

6. Cómo medimos y comparamos

6.1. El tiempo que queda en el CSV

El programa llama a `MPI_Barrier` antes de iniciar `MPI_Wtime`. En matrices, el intervalo incluye distribución, cálculo, recolección y barrera final. En trapezio, incluye el cálculo local y la reducción del resultado. Se informa el máximo de los intervalos locales de los procesos.

El tiempo `T_Total` **no incluye** iniciar `mpirun`, arrancar por SSH, `MPI_Init`, preparar las matrices ni calcular los checksums finales. Ambas comparaciones usan `T_Total`. La segunda batería también guarda `wall_time_s` como metadato, pero no lo usa para calcular speedup.

```

1 MPI_Barrier(MPI_COMM_WORLD);
2 double t0 = MPI_Wtime();
3 /* distribuir, calcular y reunir, segun el programa */
4 double t_total = MPI_Wtime() - t0;
5 MPI_Reduce(&t_total, &total_max, 1, MPI_DOUBLE,
6           MPI_MAX, 0, MPI_COMM_WORLD);

```

El fragmento resume el mecanismo. Los fuentes completos copiados junto al informe conservan los detalles de cada programa.

6.2. Speedup, eficiencia y comparación entre redes

Para el mismo problema y la misma red:

$$S(P) = \frac{T(1)}{T(P)}, \quad E(P) = \frac{S(P)}{P} 100\%, \quad R(P) = \frac{T_{\text{WiFi}}(P)}{T_{\text{Ethernet}}(P)}.$$

$S > 1$ indica mejora frente a un proceso; $S < 1$, empeoramiento. E expresa la mejora respecto de la ideal $S = P$. $R > 1$ significa que Wi-Fi tardó más. Cada red usa su propia medición $T(1)$; no se mezclan las referencias.

Para observar solo la ampliación de uno a cuatro nodos completos también usamos $T(24)/T(96)$. Esa razón no es el speedup respecto de un proceso. En trapezio grande por cable vale aproximadamente 4.01.

6.3. Por qué las fases no se suman

`T_Dist`, `T_Calc` y `T_Gather` son máximos calculados por separado. Pueden corresponder a procesos diferentes y contener esperas. Mientras un proceso ya entró a reunir resultados, otro aún puede estar recibiendo datos o calculando. Por eso **no apilamos las fases ni calculamos porcentajes sumándolas**. Tampoco interpretamos `T_Reduce` como latencia pura: puede incluir espera al proceso que termina de calcular más tarde.

6.4. Comprobaciones de consistencia

En la primera batería se cotejaron las 50 líneas de resultado con sus CSV. Sus 30 ejecuciones de matrices presentan las mismas huellas para cada tamaño, a la precisión impresa. En la segunda se cotejaron las 75 líneas y sus huellas. Son verificaciones de consistencia, no una demostración de igualdad elemento a elemento. Los scripts de medición extraen el tiempo; la revisión de huellas se hizo al generar este informe y se repite en la auditoría de la sección 16.

7. Qué mejoró al trabajar por bloques

El bucle `ikj` recorre B completa por cada fila local de A . Para $N = 3072$, B ocupa 75.497472 MB, un volumen mayor que los 36 MB de L3 documentados. El kernel `tilled` modifica el orden de recorrido para reutilizar paneles de B y teselas de C antes de pasar al siguiente bloque.

7.1. Evidencia histórica en una sola computadora

La siguiente tabla usa las **medianas de tres ejecuciones locales** de `resultados_bloques.json`, para $N = 3072$. Es un antecedente del cambio de kernel; no pertenece a las 50 mediciones de la primera batería.

P	1D-ikj (s)	1D-tilled (s)	2D-tilled (s)
1	7.150618	3.934298	3.927859
4	5.118120	1.101255	1.105275
8	5.297588	0.589795	–
16	5.483845	0.474050	0.381260
24	5.571556	0.503265	–

Con 16 procesos, pasar de 1D-ikj a 1D-tilled reduce el tiempo de 5.484 s a 0.474 s: aproximadamente 11.57 veces. El mejor tiempo 2D-tilled de esta tabla es 0.381 s. Estos datos muestran que la organización del cálculo local importa incluso cuando no hay red entre computadoras.

7.2. Qué significa la estimación de 232 GB

Si se supone que por cada fila se vuelve a leer B desde RAM, el volumen de lecturas es:

$$V_B \approx N(8N^2) = 8N^3.$$

Para $N = 3072$, esta cuenta da aproximadamente **231.93 GB**. Es un **modelo de accesos**, no un contador medido de tráfico de RAM. La reutilización real depende de caché, procesos concurrentes y comportamiento del procesador. No se registraron contadores de rendimiento que permitan afirmar que la RAM estuvo exactamente al 97 % de utilización.

El módulo DDR5 documentado ofrece un máximo teórico de 44.8 GB/s. El estancamiento del bucle original y la mejora con bloques son compatibles con una limitación de memoria; no convierten ese máximo teórico en ancho de banda observado.

Optimizar el cálculo local no elimina el tráfico de la partición 1D. `1d ikj` y `1d tiled` ejecutan las mismas colectivas sobre matrices del mismo tamaño. La reducción de MB de 1D a 2D pertenece al cambio de partición y comunicación, no al uso de teselas por sí solo.

8. Primera batería: resultados del trapecio

En las tablas, T_E es Ethernet, T_W es Wi-Fi, S_E y S_W son speedups respecto de un proceso. Cada celda de tiempo proviene de una ejecución.

8.1. Problema pequeño: $n = 10^8$

P	T_E (s)	T_W (s)	S_E	S_W	T_W/T_E
1	0.066698	0.067350	1.00	1.00	1.01
24	0.003926	0.005561	16.99	12.11	1.42
48	0.002061	0.018343	32.36	3.67	8.90
72	0.001983	0.089785	33.63	0.75	45.28
96	0.001661	0.146423	40.16	0.46	88.15

Con Ethernet, el tiempo disminuye hasta 0.001661 s con 96 procesos. En Wi-Fi, el mejor tiempo medido es con 24 procesos en un nodo: 0.005561 s. Al pasar a 96 procesos, la configuración Wi-Fi tarda 0.146423 s y resulta más lenta que un proceso. El volumen del resultado es pequeño, pero la coordinación y las esperas pesan mucho frente a un cálculo tan corto.

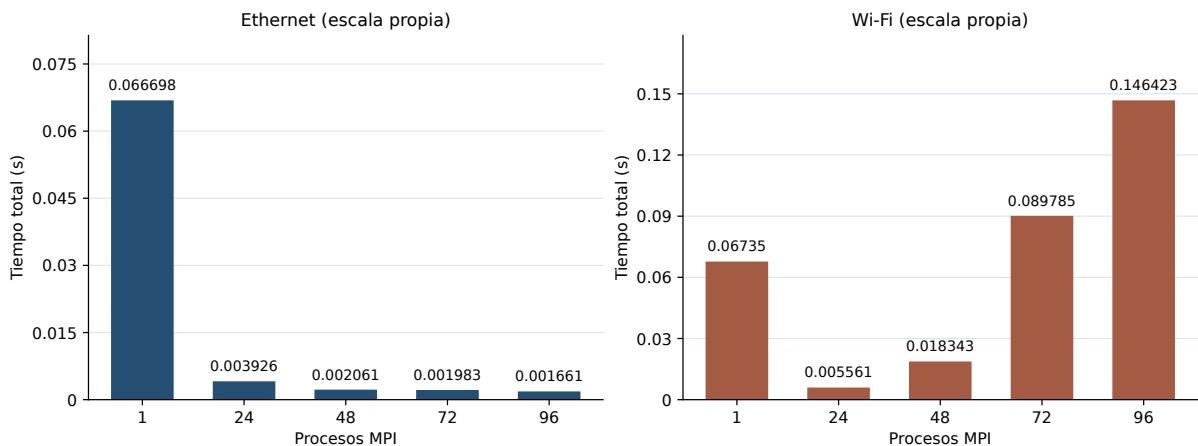


Figura 1: Trapecio con 10^8 intervalos. Una ejecución por barra; valores en segundos. Las escalas verticales son diferentes.

8.2. Por qué la Wi-Fi casi no importa con 24 procesos

En $P = 1$ y $P = 24$, todos los procesos calculan en el servidor. El nombre de la red identifica las opciones del lanzador, pero el trabajo MPI de estos casos no necesita ir a otro nodo. Las diferencias pequeñas entre estas dos configuraciones no demuestran una ventaja de una interfaz para el cálculo local.

8.3. Problema grande: $n = 10^{11}$

P	T_E (s)	T_W (s)	S_E	S_W	T_W/T_E
1	66.113517	66.526926	1.00	1.00	1.01
24	3.869576	4.045816	17.09	16.44	1.05
48	1.927789	2.016062	34.29	33.00	1.05
72	1.285578	1.368964	51.43	48.60	1.06
96	0.964298	1.053217	68.56	63.17	1.09

Con Ethernet, pasar de 24 a 48, 72 y 96 procesos da aproximadamente 2.01, 3.01 y 4.01 veces la velocidad del nodo completo. Es un escalado casi proporcional al número de nodos en estas mediciones. Frente a un proceso, el speedup final es 68.56 y la eficiencia es 71.42 %.

Con Wi-Fi, el resultado de 96 procesos es 1.053217 s, un speedup de 63.17 y una eficiencia de 65.80 %. La razón Wi-Fi/Ethernet es 1.09, muy inferior a la de trapezio pequeño. El cálculo largo amortiza el costo de la comunicación.

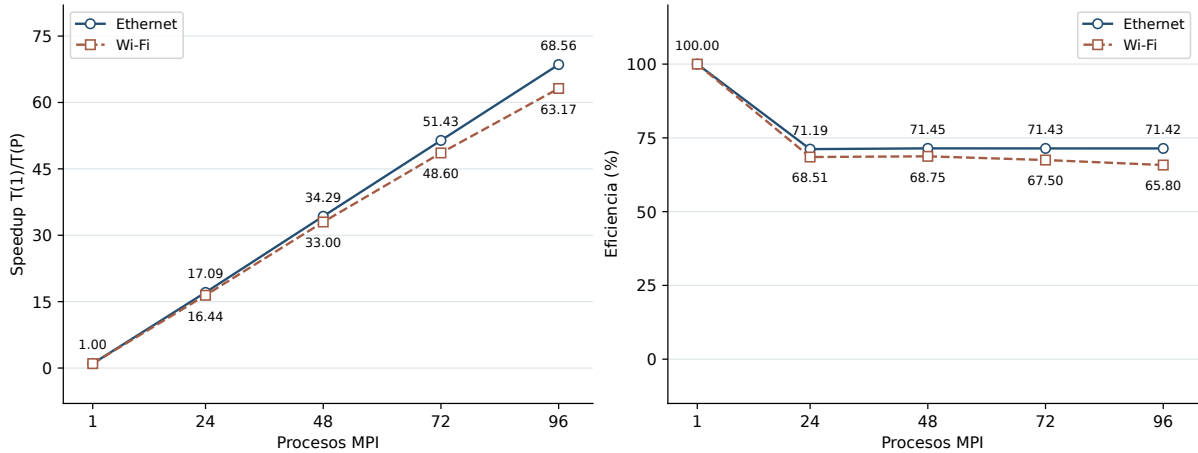


Figura 2: Speedup y eficiencia del trapezio grande. Las cifras anotadas se calcularon con los CSV; no hay barras de incertidumbre porque se hizo una ejecución por configuración.

8.4. Por qué la eficiencia no llega al 100 %

El clúster combina núcleos P y E, reparte casi el mismo número de trapezios a cada proceso y espera el resultado de todos. La diferencia de rendimiento de los núcleos puede producir desbalance; también intervienen frecuencia, instrucciones ejecutadas, coordinación y carga del sistema.

La eficiencia medida de 71.42 % **no se puede atribuir exactamente a la relación de frecuencias P/E**. Las frecuencias máximas no son las frecuencias sostenidas y no resumen por sí solas el rendimiento. Las mediciones muestran el efecto final; no aíslan cuánto corresponde a cada causa.

9. Primera batería: multiplicación de matrices

Todas estas filas usan **1D-tiled con bloque 64**. $P = 1$ y $P = 24$ se ejecutan en un solo nodo; los siguientes puntos utilizan dos, tres y cuatro nodos.

9.1. Matriz 1024×1024

P	T_E (s)	T_W (s)	S_E	S_W	T_W/T_E
1	0.145456	0.147472	1.00	1.00	1.01
24	0.032678	0.034179	4.45	4.31	1.05
48	0.347345	5.549577	0.42	0.03	15.98
72	0.567399	9.269884	0.26	0.02	16.34
96	0.772748	14.113333	0.19	0.01	18.26

9.2. Matriz 2048×2048

P	T_E (s)	T_W (s)	S_E	S_W	T_W/T_E
1	1.199625	1.244563	1.00	1.00	1.04
24	0.193562	0.199079	6.20	6.25	1.03
48	1.252217	21.085757	0.96	0.06	16.84
72	1.982867	31.687304	0.60	0.04	15.98
96	2.788212	46.322868	0.43	0.03	16.61

9.3. Matriz 3072×3072

P	T_E (s)	T_W (s)	S_E	S_W	T_W/T_E
1	3.816321	3.904030	1.00	1.00	1.02
24	0.512022	0.553372	7.45	7.05	1.08
48	2.815416	44.256782	1.36	0.09	15.72
72	4.352181	65.984071	0.88	0.06	15.16
96	6.174707	101.042337	0.62	0.04	16.36

En los tres tamaños, **24 procesos es el mejor punto de la primera batería**. Para $N = 3072$, Ethernet baja de 3.816321 s a 0.512022 s: mejora 7.45 veces. Al pasar a 48 procesos el tiempo aumenta a 2.815416 s, y con 96 llega a 6.174707 s, incluso por encima de la referencia de un proceso.

La misma tendencia es más fuerte por Wi-Fi: de 0.553372 s con 24 procesos se pasa a 101.042337 s con 96. Para ese tamaño, usar cuatro nodos tarda aproximadamente **182.59 veces** lo que tarda un nodo con 24 procesos bajo la configuración Wi-Fi.

9.4. Más procesos pueden aumentar el tiempo

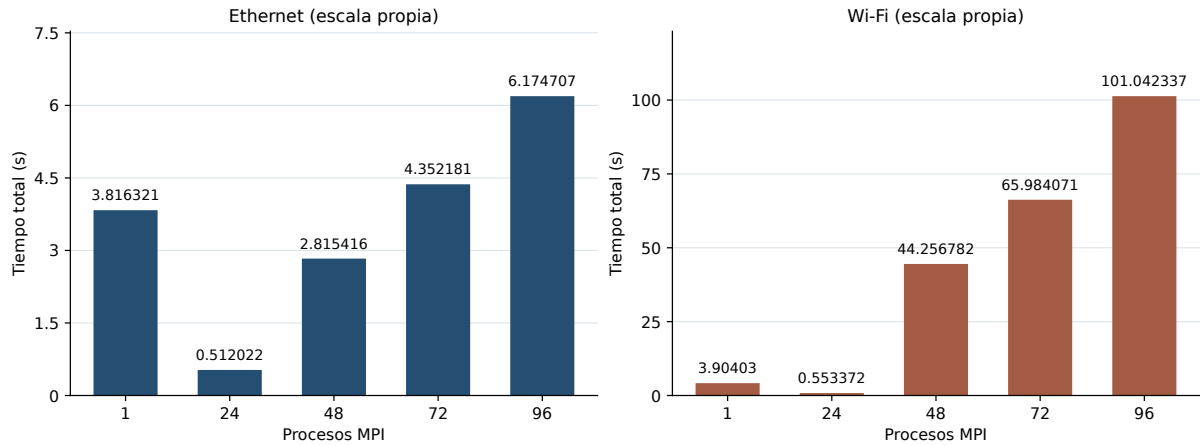


Figura 3: Matrices de $N = 3072$. Escalas verticales distintas para Ethernet y Wi-Fi. Las barras muestran T_{Total} , incluyendo las comunicaciones del algoritmo.

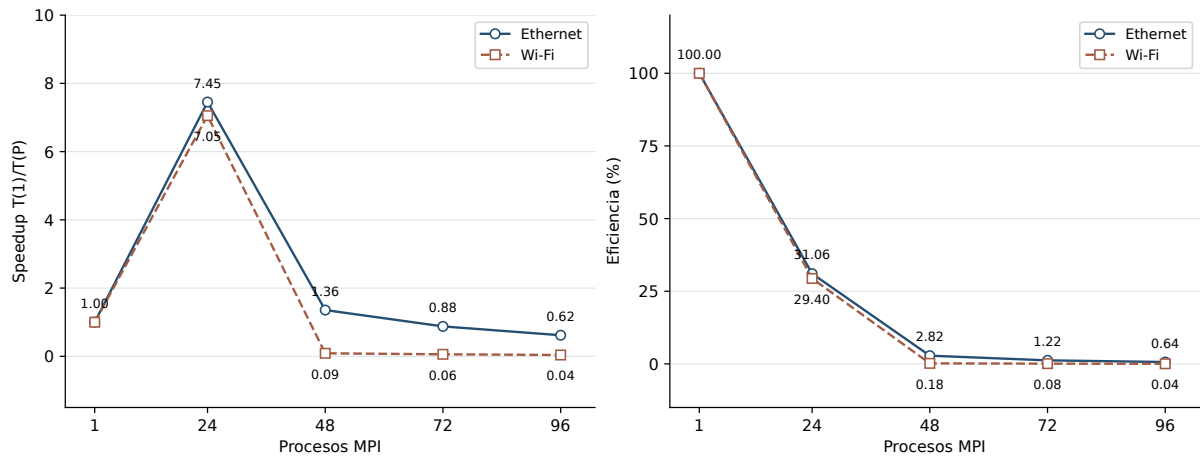


Figura 4: Speedup y eficiencia para $N = 3072$. El speedup inferior a 1 de algunos puntos indica que son más lentos que un proceso.

Con 96 procesos, Ethernet tiene $S = 0.618$ y una eficiencia de aproximadamente 0.644 %. Wi-Fi tiene $S = 0.0386$ y eficiencia de 0.0402 %. Son resultados desfavorables, pero responden la pregunta experimental: **en estos tamaños y con esta partición, la comunicación entre nodos cuesta más que el cálculo ahorrado.**

9.5. Dónde se va el tiempo

Para $N = 3072$, los logs permiten mirar las fases:

P	Red	Distribuir (s)	Calcular (s)	Reunir (s)	Total (s)
24	Ethernet	0.198404	0.304399	0.055386	0.512022
24	Wi-Fi	0.216094	0.344965	0.083387	0.553372
96	Ethernet	5.998111	0.104963	3.150420	6.174707
96	Wi-Fi	95.931070	0.100977	58.381650	101.042337

Con 96 procesos, el cálculo máximo es aproximadamente 0.105 s por cable y 0.101 s por Wi-Fi. En cambio, la distribución llega a 5.998 s por cable y 95.931 s por Wi-Fi. El total de Wi-Fi es **16.36 veces** el de Ethernet.

El cálculo medido es parecido, mientras la distribución y la recolección son mucho mayores por Wi-Fi. Esa es la evidencia directa del cuello de botella de comunicación. Cada fase también puede incorporar esperas; estas mediciones no separan ancho de banda, sincronización y retransmisiones. Como se explicó antes, **las columnas no son sumables**.

9.6. Verificación de las matrices calculadas

Para cada tamaño de la primera batería, sus diez ejecuciones (cinco valores de P por dos redes) imprimen la misma suma de la diagonal y la misma suma ponderada:

N	Suma diagonal	Suma ponderada	Coinciden
1024	2453667.02	1.758790 e+10	10/10
2048	9814670.59	1.407031 e+11	10/10
3072	22083010.24	4.748730 e+11	10/10

La coincidencia permite detectar errores evidentes de reparto o colocación. No sustituye una comparación elemento a elemento ni garantiza que no exista un error compartido por todas las ejecuciones.

En una exposición, el contraste más claro es $N = 3072$ con $P = 24$ y $P = 96$. Dentro de un nodo se aprovechan varios núcleos sin mover matrices por la red. Al utilizar cuatro nodos, cada proceso calcula menos, pero aparecen transferencias y esperas mucho mayores.

10. Segunda batería: resultados ampliados del 25 de septiembre

El documento de trabajo `fuentes/INFORME_ANALISIS_EXPANDIDO_96.md` motivó esta ampliación. Para las cifras se usan sus archivos originales, copiados en `datos/ampliada/`. La batería realizó **75 ejecuciones únicas**: 40 por Ethernet y 35 por Wi-Fi. Se conservan los cinco valores de $P = 1, 24, 48, 72, 96$. Trapecio mantiene $n = 10^8$ y 10^{11} ; matrices añade $N = 512$ y 4096 a ambas redes, y $N = 6144$ solo a Ethernet. Los lanzadores y el programa 1D-tiled son los mismos de la primera batería.

El CSV consolidado y los dos CSV de tiempos coinciden con las 75 líneas de resultado de los 75 logs. También coinciden la traza y la suma ponderada de cada tamaño de matriz entre sus configuraciones, a la precisión impresa. Esta comprobación detecta discrepancias entre corridas, pero no sustituye una verificación elemento a elemento. Como se hizo una ejecución por configuración, las diferencias pequeñas pueden depender de las condiciones de ese momento.

10.1. Trapecio: el trabajo largo sigue escalando

n	Red	T_1 (s)	T_{24} (s)	T_{96} (s)	S_{96}
10^8	Cable	0.066984	0.003921	0.001508	44.42
10^8	Wi-Fi	0.067826	0.005612	0.115863	0.59
10^{11}	Cable	66.488999	3.842393	0.974949	68.20
10^{11}	Wi-Fi	66.857550	3.937623	1.135211	58.89

Con 10^{11} trapecios, los 96 procesos tardaron 0.974949 s por Ethernet y 1.135211 s por Wi-Fi. Las aceleraciones respecto a un proceso de la *misma red y la misma batería* fueron 68.20 y 58.89. Con 10^8 por Wi-Fi, 96 procesos tardaron 0.115863 s frente a 0.067826 s con uno: para esta carga breve el coste de comunicar y sincronizar anuló la ventaja del paralelismo.

10.2. Matrices: un nodo completo sigue siendo el mejor punto

N	Cable T_1	Cable T_{24}	Cable T_{96}	Wi-Fi T_{24}	Wi-Fi T_{96}
512	0.019	0.007	0.211	0.007	4.299
1024	0.145	0.034	0.760	0.035	13.648
2048	1.198	0.200	2.787	0.215	47.759
3072	3.889	0.547	6.148	0.553	99.945
4096	9.914	1.325	11.042	1.410	178.957
6144	32.888	3.839	25.037	–	–

El menor tiempo de cada tamaño medido corresponde a $P = 24$, es decir, los 24 procesos dentro de **servidor**. Para $N = 3072$, $P = 24$ tardó 0.547426 s por cable y 0.552753 s por Wi-Fi; $P = 96$ tardó 6.148198 y 99.944942 s. Para $N = 4096$, la corrida Wi-Fi con 96 procesos llegó a 178.957201 s.

La matriz de 6144×6144 se midió solo por Ethernet: 32.887841 s con un proceso, 3.839080 s con 24 y 25.036958 s con 96. Así, 96 procesos superaron al proceso único por 1.31 veces, pero tardaron **6.52 veces más** que el nodo completo. El punto de cruce frente a un proceso se sitúa entre los tamaños 4096 y 6144 observados; no se midió un tamaño intermedio ni se encontró un cruce frente a 24 procesos.

La cifra de 6.52 veces procede de una sola corrida por configuración. La campaña de optimización repitió $N = 6144$ tres veces y obtuvo un cociente de medianas de 4.77, con 24 procesos entre 4.998 y 7.343 s. Ambas fuentes coinciden en la dirección del resultado, pero la magnitud depende de la corrida y del binario; la sección 15 explica esa diferencia.

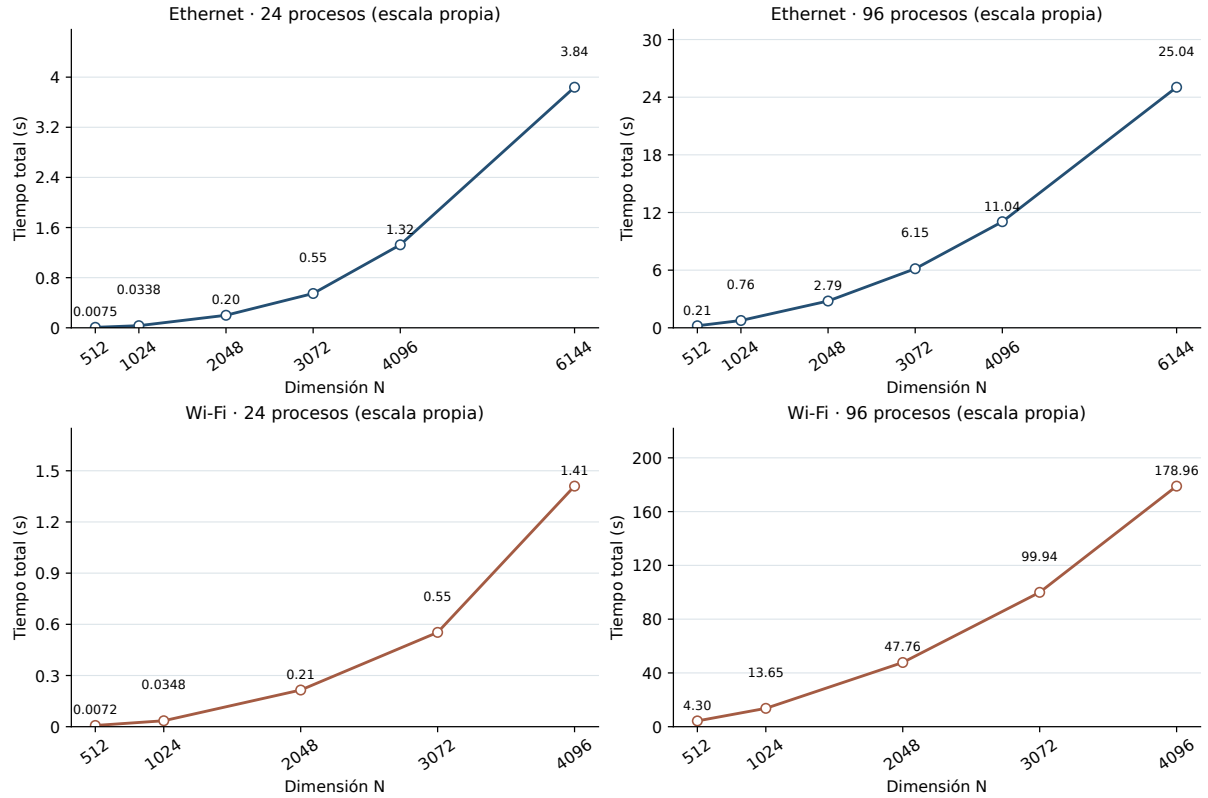


Figura 5: Segunda batería de matrices. Cada panel usa escala vertical lineal propia y anota los tiempos medidos en segundos; no se comparan alturas de paneles con escalas diferentes. Una ejecución por punto.

10.3. Qué miden los MB nuevos

El script `ejecutar_bateria_expandida.py` lee `tx_bytes` y `rx_bytes` en la interfaz de *servidor* antes y después de cada lanzamiento. La columna `mb_real_master` es la **suma TX+RX de esa interfaz durante toda la ejecución del comando**, incluido el arranque de MPI. Puede contener tráfico ajeno. No incluye las interfaces de los tres workers ni separa `MPI_Bcast`, `MPI_Scatterv` y `MPI_Gatherv`.

N	Red	M (MB)	Modelo (MB)	Servidor TX+RX (MB)
3072	Cable	75.50	339.74	516.72
3072	Wi-Fi	75.50	339.74	500.88
4096	Cable	134.22	603.98	917.87
4096	Wi-Fi	134.22	603.98	887.70
6144	Cable	301.99	1358.95	2065.61

Para $N = 3072$ y $P = 96$, el modelo ideal de carga útil da 339.74 MB, mientras la interfaz del servidor registró 516.72 MB por Ethernet y 500.88 MB por Wi-Fi en TX+RX. **Modelo y contador tienen alcances distintos**; la diferencia no identifica por sí sola el algoritmo colectivo. Los 357–358 MB de otra serie histórica sumaban únicamente TX en varios nodos y con otra configuración, por lo que no se deben equiparar con estos números.

Los ping se tomaron una vez *antes* de iniciar las dos fases de la batería, no durante cada prueba:

Nodo	Ethernet (ms)	Wi-Fi (ms)
workers1	1.149	52.320
workers2	1.272	123.587
workers4	1.212	95.978

Sirven como contexto de red en ese instante; no explican por sí solos la variación de una ejecución posterior.

10.4. Exploración de núcleos P y E

El documento ampliado describe un microbenchmark local de 24 núcleos y un prototipo de trapecio con reparto ponderado. Se conservan sus fuentes en `fuentes/exploratorios/`. El borrador informa una razón de rendimiento P/E de 1.2658 para su cálculo de trapecio y tiempos mejores con reparto ponderado. No se conservaron las salidas individuales de esas pruebas junto a la batería de 75 logs, y el prototipo no se usó en ella. Por eso esta línea se presenta como **exploratoria**, sin convertir la reducción de espera declarada en una conclusión reproducida. La campaña posterior sí repitió un trapecio simétrico y otro asimétrico tres veces cada uno (tabla 2): la mejora no fue consistente y el desbalance registrado no bajó de forma sistemática.

Además, el prototipo exploratorio `fuentes/exploratorios/trapecio_asimetrico.c` recorre índices de 0 a $n - 1$ y nunca evalúa el extremo $i = n$: le falta el término final $f(1)h/2$ de la regla del trapecio. La versión usada en la campaña (`src/trapecio_asimetrico.c`) ya incorpora esa corrección.

El borrador también propone que 128 GB agregados permitirían multiplicar matrices mucho mayores con el programa actual. Sin embargo, en la implementación 1D **cada proceso reserva la matriz B completa** y el rango 0 conserva A , B y C . Esa memoria no se combina automáticamente entre nodos. La batería más grande ejecutada fue $N = 6144$; no se midió la capacidad ni el tiempo para $N \approx 45\,500$.

11. La madrugada: 906 segundos y el costo de difundir B

11.1. Caso central: el comando y el registro de las 00:58

La imagen que motivó esta revisión resume una ejecución concreta. `repetir_profiling_cluster.py` lanzó `/tmp/matrix_profiling` con $N = 3072$, ocho procesos y dos procesos en cada una de las cuatro computadoras. Su comando efectivo fue:

```
1 mpirun -H servidor:2,workers1:2,workers2:2,workers4:2 -np 8 \
2   --mca btl_tcp_if_include 10.7.134.0/23 \
3   --mca oob_tcp_if_include 10.7.134.0/23 \
4   /tmp/matrix_profiling 3072
```

El registro conservado en `datos/historicos/repetir_profiling_cluster.log` dice:

```
1 [2026-09-24T00:58:05] rep 1/2 N=3072 cluster_8p ...
2 -> Total=906.35s Calc=1.74s Bcast=889.74s
```

La observación decisiva: el máximo de `MPI_Bcast(B)` fue **889.744 s**, frente a **906.351 s** totales y **1.735 s** de cálculo. El cociente **98.17 %** indica que la espera registrada en la difusión dominó esta corrida. El programa original usaba franjas 1D y kernel `ikj` sin teselas. El comando seleccionaba la subred Wi-Fi, pero no fijaba afinidad ni componentes BTL; eso *no demuestra* que toda la comunicación local viajara por TCP.

Entre las 00:37 y 01:37 del 24/09 se repitieron las pruebas del mismo ejecutable. El archivo `resultados_profiling_repeticion.json` separa los máximos de `MPI_Bcast` y `MPI_Scatterv`. Para $N = 3072$:

P	Rep.	Inicio	T_{Bcast} (s)	T_{Calc} (s)	T_{Total} (s)
4	1	00:53:08	276.244	2.645	290.104
4	2	01:24:59	87.828	2.448	101.805
8	1	00:58:05	889.744	1.735	906.351
8	2	01:26:48	306.769	1.737	322.513
16	1	01:13:19	289.159	0.786	300.886
16	2	01:32:18	294.318	0.964	308.340

La segunda repetición de la misma configuración tardó 322.513 s y tuvo un tiempo de cálculo casi idéntico. Esto localiza la variación en la comunicación y las esperas, no en un aumento del trabajo matemático.

Esta serie no registró MB transmitidos. Por ello no le asignamos los 1112 MB observados después con otro ejecutable y 16 procesos. La fuente tampoco permite sumar máximos de `Bcast` y `Gather`: en la corrida de 906 s, ambos contienen esperas de procesos distintos.

11.2. Segunda serie: medir tiempo y tráfico

Entre las 01:45 y 02:18 se compararon 1D-ikj y 2D-tiled con el ejecutable `mpi_matrix_2d`. El script leyó `tx_bytes` antes y después en los cuatro nodos, y guardó las diferencias en `resultados_bloques.json`. Son los datos de la página siguiente.

La evidencia respalda que hubo mucho tráfico y esperas de comunicación. Congestión, interferencia, reintentos y cambios de tasa de la Wi-Fi son causas posibles. No se registraron contadores suficientes para atribuir cada corrida a colisiones concretas ni para afirmar que se observó un mecanismo específico de *backoff*.

11.3. Los MB medidos durante la madrugada

Todos los registros de esta tabla corresponden a $N = 3072$, cuatro nodos y Wi-Fi.

P	Versión	Rep.	Total (s)	MB TX	MB/s*
4	1D-ikj	1	55.842	342.34	6.13
4	1D-ikj	2	58.540	342.65	5.85
4	2D-tiled	1	85.681	323.87	3.78
4	2D-tiled	2	61.799	324.43	5.25
16	1D-ikj	1	338.847	1112.24	3.28
16	1D-ikj	2	322.789	1111.50	3.44
16	2D-tiled	1	179.684	423.41	2.36
16	2D-tiled	2	66.172	422.35	6.38

*Cociente entre la suma de MB transmitidos por las interfaces y T_{Total} . Es un indicador agregado de la ejecución, no la velocidad física de un enlace.

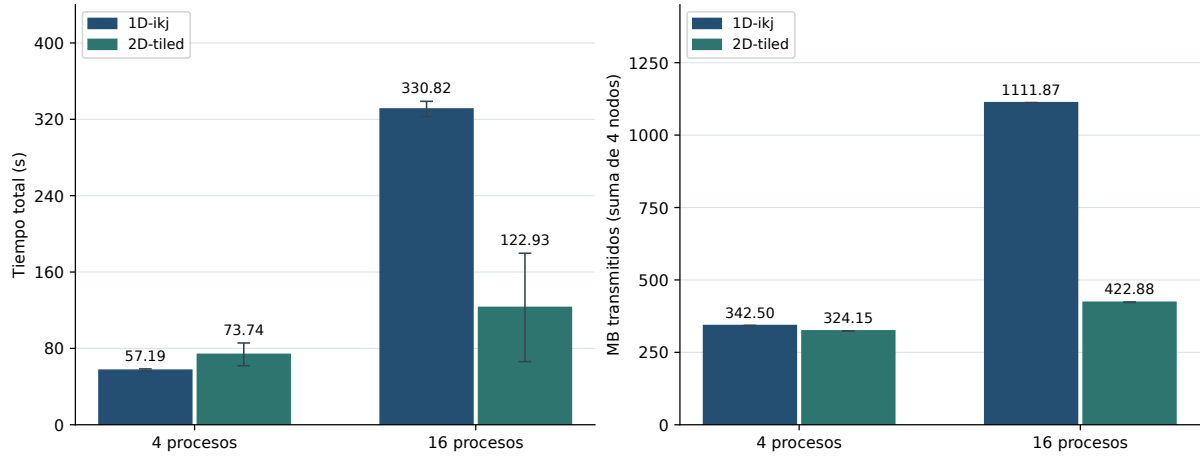


Figura 6: Medianas de dos ejecuciones históricas. Las barras de error indican mínimo y máximo, no intervalos de confianza. Los MB provienen de los contadores de las cuatro interfaces.

Con 16 procesos, 1D-ikj transmitió 1112.24 y 1111.50 MB. 2D-tiled transmitió 423.41 y 422.35 MB: aproximadamente **2.63 veces menos**. La mediana del tiempo pasó de 330.818 s a 122.928 s. Sin embargo, 2D-tiled varió entre 66.172 y 179.684 s con volúmenes casi iguales. **La cantidad de bytes explica parte del costo, pero no toda la variación temporal.**

11.4. Comparación histórica del mediodía

La serie de las 11:25–11:52 comparó ambas redes, con dos repeticiones Ethernet y una Wi-Fi. La tabla muestra medianas en Ethernet y la ejecución única en Wi-Fi, siempre con $N = 3072$ y cuatro nodos.

P	Versión	Cable (s)	Cable MB	Wi-Fi (s)	Wi-Fi MB
4	1D-ikj	4.388	357.31	56.154	342.02
4	1D-tiled	3.713	357.27	70.691	342.15
4	2D-tiled	3.170	337.75	51.559	323.30
16	1D-ikj	6.610	1152.07	198.721	1120.43
16	1D-tiled	5.467	1151.70	188.147	1125.16
16	2D-tiled	3.550	437.06	91.905	423.05
64	1D-ikj	4.602	357.91	–	–
64	1D-tiled	3.153	357.79	–	–
64	2D-tiled	5.592	677.95	–	–

Aquí aparecen dos observaciones útiles:

1. Con 16 procesos, 1D-tiled sigue moviendo unos 1125 MB por Wi-Fi, aunque optimice la caché. Su tiempo es 188.147 s. La mejora local no elimina la comunicación de 1D.
2. Con 64 procesos por cable, 1D-tiled registra unos 357.79 MB; con 16 procesos por cable registra unos 1151.70 MB. Ambos lanzamientos históricos carecían de la selección explícita `btlself, vader, tcp`. Por tanto, el cambio de volumen no se puede atribuir únicamente a haber añadido esa opción en los scripts actuales.

Los 357–358 MB son mediciones de esta serie histórica, no de la primera batería de 96 procesos. La primera batería conserva tiempos y logs, pero no muestrea los contadores de red. La segunda sí registra TX+RX del servidor, una magnitud diferente. Tampoco se deben

presentar los 5.29–5.64 s de 1D-tiled con 16 procesos como repeticiones Wi-Fi: esos valores son de Ethernet.

Al comparar la madrugada con el mediodía, 1D-ikj con 16 procesos pasó de una mediana de 330.818 s a una corrida Wi-Fi de 198.721 s. El volumen fue parecido (unos 1112 frente a 1120 MB), lo que es compatible con variaciones del rendimiento efectivo de la red. No se controlaron las condiciones del punto de acceso para atribuir la diferencia a una causa única.

12. Qué cambió en las instrucciones de lanzamiento

12.1. Comando histórico

El script `repetir_profiling_cluster.py` construía la siguiente lista para 16 procesos. BIN apuntaba a `/tmp/matrix_profiling`:

```
1 ['mpirun', '-H',  
2 'servidor:4,workers1:4,workers2:4,workers4:4',  
3 '-np', '16',  
4 '--mca', 'btl_tcp_if_include', '10.7.134.0/23',  
5 '--mca', 'oob_tcp_if_include', '10.7.134.0/23',  
6 BIN, str(n)]
```

Los nombres resolvían a las IP Wi-Fi. El comando seleccionaba la subred TCP, pero dejaba implícitas las decisiones de asignación, afinidad y componentes de transporte. Para la prueba de 906 s, los hosts tenían `:2` y el total era `-np 8`. En la serie que midió 1112 MB se añadió al nuevo ejecutable la selección `3072 1d ikj 64`.

12.2. Comando actual

El lanzador Wi-Fi actual fija la asignación, la afinidad y los transportes permitidos. Su estructura real es:

```
1 mpirun -H "$hosts" -np "$procesos" \  
2 --map-by slot --rank-by slot --bind-to core \  
3 --report-bindings \  
4 --mca plm_rsh_agent "ssh -F $DIR/ssh_config_wifi" \  
5 --mca btl self,vader,tcp \  
6 --mca btl_tcp_if_include 10.7.134.0/23 \  
7 --mca oob_tcp_if_include 10.7.134.0/23 \  
8 "${programa[@]}"
```

Para matrices con 96 procesos, `hosts` contiene las cuatro IP Wi-Fi con `:24`, y `programa` es un arreglo Bash con el ejecutable y los argumentos `3072 1d tiled 64`. Las comillas y la expansión `"${programa[@]}"` preservan los argumentos separados.

En Ethernet se usa el mismo patrón con las IP `10.7.50.*`, la subred `10.7.50.0/24` y `ssh_config`. El anexo contiene los comandos completos de uso y copias de los scripts.

12.3. Qué significa cada opción

Opción	Efecto
-H y -np	Indican hosts/slots disponibles y número total de procesos.
--map-byslot	Coloca procesos por slots, llenando los nodos según el orden indicado.
--rank-byslot	Ordena los rangos MPI por slots.
--bind-to-core	Restringe cada proceso al núcleo asignado.
--report-bindings	Imprime la afinidad usada para comprobarla en los logs.
--mcabtlself, vader, tcp	Restringe los componentes BTL permitidos: comunicación consigo mismo, memoria compartida local y TCP entre nodos.
btl_tcp_if_include	Restringe las interfaces o subredes usadas por el BTL TCP.
oob_tcp_if_include	Restringe la red TCP del canal de control de Open MPI.
plm_rsh_agent	Selecciona SSH y su archivo de configuración para iniciar procesos remotos.

12.4. Memoria compartida no demuestra difusión jerárquica

La selección **vader** es una decisión sobre **cómo transportar un mensaje entre procesos locales**. El algoritmo de **MPI_Bcast** decide **qué procesos envían a cuáles**. Son decisiones distintas. Open MPI 4.x puede usar memoria compartida por defecto aunque el comando no mencione **vader** [2].

Por eso no basta con llamar al comando nuevo “jerárquico” y al antiguo “plano”. Las dos líneas, por sí solas, no prueban que antes se enviara todo por TCP local ni que ahora viaje exactamente una copia de B por nodo. En el código actual cada proceso sigue reservando su propia matriz B ; no hay una ventana MPI de memoria compartida para que todos usen un único arreglo.

Una comprobación adicional del código oficial de Open MPI 4.1.6 muestra que, *si se usa la decisión fija del componente tuned*, para mensajes de 75.5 MB se elegiría **basic_linear** con 8 procesos, **scatter_allgather** con 16 y **knomial** con 96 [3]. Esto hace plausible que P cambie el patrón de tráfico, pero **no demuestra qué componente y algoritmo se activaron en cada ejecución histórica**: no se guardó esa traza.

La campaña de optimización aporta evidencia medida, aunque indirecta. Con $N = 3072$, la variante 1D global transmitió 358.0 MB agregados con $P = 64$ y 755.5 MB con $P = 96$, mientras la difusión jerárquica explícita con $P = 96$ transmitió 358.9 MB. El patrón por defecto, por tanto, sí cambia con P y con 96 procesos mueve unas 2.1 veces más datos que una difusión por nodo. En la difusión aislada con 16 procesos, la colectiva por defecto y la selección forzada **scatter_allgather** dieron el mismo TX (1006.5 MB por Wi-Fi), lo que es compatible con la tabla de decisión citada. Sigue sin registrarse qué algoritmo se eligió en cada ejecución.

13. Cómo interpretar los MB transmitidos

13.1. Medición real y modelo de tráfico

Los scripts históricos leen el contador `/sys/class/net/INTERFAZ/statistics/tx_bytes` en cada nodo antes y después de ejecutar. Calculan:

$$V_{\text{TX}} = \frac{1}{10^6} \sum_{h=1}^4 (\text{TX}_{h,\text{después}} - \text{TX}_{h,\text{antes}}).$$

Son bytes contabilizados por las interfaces durante esa ventana. Pueden incluir tráfico ajeno y de control; no aíslan cada colectiva ni representan todo el uso del aire, los reenvíos del punto de

acceso o todos los reintentos de radio. Tampoco se debe comparar el cociente agregado V_{TX}/T de cuatro nodos con el máximo de un único enlace.

13.2. Cuánto pesa una matriz

$$M = 8N^2 \text{ bytes.}$$

N	Tamaño de una matriz (MB)
1024	8.388608
2048	33.554432
3072	75.497472

Con cuatro nodos equilibrados, el proceso 0 conserva una cuarta parte de las filas de A y C . Una contabilidad ideal de los datos que cruzan entre nodos es:

Supuesto para 1D	Expresión	MB con $N = 3072$
Repartir A y reunir C	$0.75M + 0.75M$	113.25
Difundir una copia de B a cada nodo remoto	$3M$	226.49
Total bajo ese supuesto	$4.5M$	339.74
Enviar una copia de B a cada rango remoto ($P = 16$)	$12M$	905.97
Total bajo el segundo supuesto	$13.5M$	1019.22

Estos son **modelos de carga útil**, sin cabeceras ni control. El segundo produce un volumen del mismo orden que los 1112 MB observados, pero esa cercanía no identifica de forma única el algoritmo. No se puede asignar toda la diferencia a un porcentaje fijo de cabeceras ni deducir un número exacto de copias a partir del total.

El modelo de 339.74 MB es compatible en orden de magnitud con los 357–358 MB históricos de ciertas configuraciones. Para la **primera batería** con 96 procesos, el tráfico medido no está disponible. La segunda registró TX+RX solo del servidor, según se explica en su sección; las cifras no son directamente comparables. Por eso no presentamos 357 MB como medición de ninguna de las dos baterías ni calculamos un ancho de banda observado con esa cifra supuesta.

14. Campaña final de optimización

Esta sección incorpora la campaña controlada ejecutada después de las dos baterías históricas. Su propósito fue comprobar qué cambios reducen el coste de comunicación observado en la corrida de 906 s y separar tres efectos que antes estaban mezclados: el algoritmo de difusión, la topología de la partición y el tamaño del problema.

14.1. Protocolo y control de calidad

Las matrices usan el kernel `tilled` con bloque 64, afinidad de un proceso por núcleo y las interfaces de red fijadas en el lanzador. La variante `1d` distribuye filas de A , realiza una difusión global de B y reúne C . La variante `1d-hier` primero difunde entre un líder por nodo y después dentro de cada nodo. SUMMA 2D se evalúa solo con $P = 64$, porque la implementación exige una malla cuadrada. El microbenchmark de Bcast transmite una matriz de 75.497 MB y comprueba una huella de 64 bits idéntica en todos los procesos. Las variantes `tuned` fuerzan el algoritmo de la colectiva con `-mca_coll_tuned_use_dynamic_rules 1` y `-mca_coll_tuned_bcast_algorithm 7` (knomial) u 8 (scatter/allgather).

Se conservaron 118 intentos válidos: cinco repeticiones para las comparaciones principales de $N = 3072$ y tres para tamaños mayores, trapecio y difusión aislada. El auditor contrasta cada fila del CSV con su log, comprueba las sumas de control de matrices, verifica el error numérico del trapecio y genera SHA256SUMS. Los intentos piloto y de humo se mantienen en el repositorio como validación operativa, pero no entran en las medianas de esta sección.

Advertencia de compilación. El archivo `entorno.json` de la campaña registra `mpicc -O3 -std=c11 -Wall -Wextra`. Las dos baterías anteriores se compilaron con `-O3 -march=native -funroll-loops`. Las comparaciones *dentro* de la campaña usan el mismo binario y son válidas; los tiempos absolutos de cálculo *no* deben compararse directamente con los de las baterías. La sección 15 cuantifica la diferencia.

14.2. Resumen de las comparaciones

La tabla 1 reúne las comparaciones que responden a las preguntas de la campaña. $\tilde{T}$ es la mediana; el cociente de TX usa las medianas del tráfico agregado. La última columna indica si los rangos mínimo-máximo de A y B están *separados* o se *solapan*. Con tres o cinco repeticiones no se estiman intervalos de confianza; un solapamiento indica que la diferencia de medianas no basta para afirmar una mejora.

Tabla 1: Comparaciones principales de la campaña de optimización. Un cociente $\tilde{T}_A/\tilde{T}_B$ mayor que 1 significa que B fue más rápido.

Comparación A / B	Red	$\tilde{T}_A$ (s)	$\tilde{T}_B$ (s)	$\tilde{T}_A/\tilde{T}_B$	TX_A/TX_B	Rangos
<i>Difusión de B en la multiplicación completa ($N = 3072$, $P = 96$, 4 nodos)</i>						
1D global / 1D jerárquica	Ethernet	6.260	3.217	1.95	2.11	separados
1D global / 1D jerárquica	Wi-Fi	101.082	51.479	1.96	2.13	separados
<i>Difusión aislada de 75.5 MB ($P = 16$)</i>						
Global / jerárquica, 4 nodos	Ethernet	4.294	2.034	2.11	4.33	separados
Global / jerárquica, 4 nodos	Wi-Fi	133.043	30.464	4.37	4.41	separados
knomial / jerárquica, 4 nodos	Ethernet	1.977	2.034	0.97	1.00	separados
knomial / jerárquica, 4 nodos	Wi-Fi	29.575	30.464	0.97	1.00	separados
Global: 4 nodos / 1 nodo	Ethernet	4.294	0.116	37.08	–	separados
<i>Partición del problema ($N = 3072$, $P = 64$, 4 nodos)</i>						
1D / SUMMA 2D	Ethernet	3.147	5.492	0.57	0.53	separados
1D / SUMMA 2D	Wi-Fi	51.728	87.314	0.59	0.51	separados
<i>Cuatro nodos frente a uno (1D global)</i>						
T_{96}/T_{24} , $N = 3072$	Ethernet	6.260	0.649	9.65	–	separados
T_{96}/T_{24} , $N = 3072$	Wi-Fi	101.082	0.666	151.70	–	separados
T_{96}/T_{24} , $N = 6144$	Ethernet	25.194	5.279	4.77	–	separados
T_{96}/T_{24} , $N = 7168$	Ethernet	35.116	7.686	4.57	–	separados
<i>Reparto del trapecio: simétrico / asimétrico</i>						
$n = 10^8$, $P = 24$	Ethernet	0.0072	0.0042	1.70	–	solapan
$n = 10^8$, $P = 96$	Ethernet	0.0020	0.0019	1.07	–	solapan
$n = 10^{11}$, $P = 24$	Ethernet	5.183	4.940	1.05	–	solapan
$n = 10^{11}$, $P = 96$	Ethernet	1.012	1.046	0.97	–	separados

14.3. Difusión jerárquica: la mejora demostrada

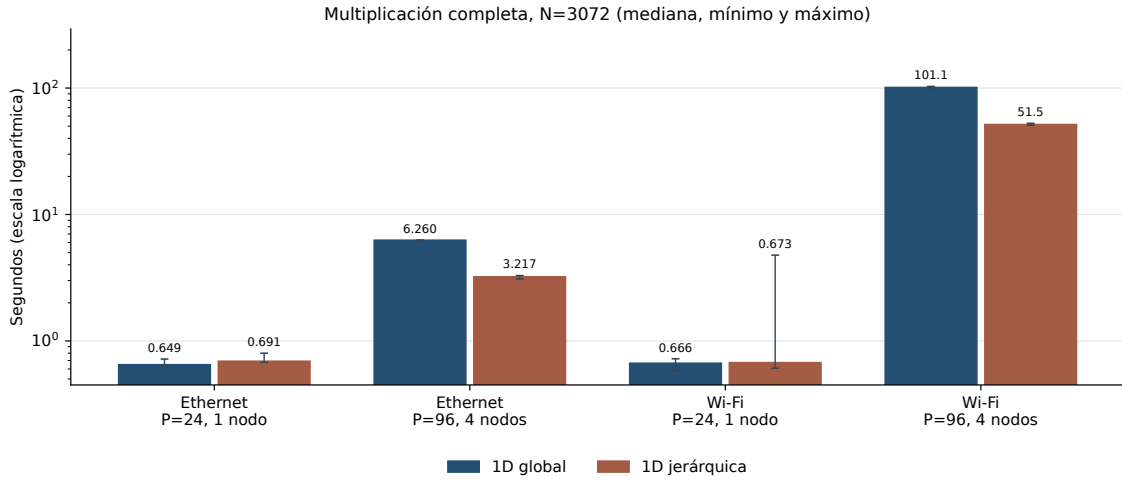


Figura 7: Multiplicación completa para $N = 3072$. Barras: mediana; líneas: mínimo y máximo de cinco repeticiones. Escala vertical logarítmica.

Con cuatro nodos y 96 procesos, la variante jerárquica redujo el tiempo total 1.95 veces por Ethernet y 1.96 por Wi-Fi, y el TX agregado 2.11 y 2.13 veces. El máximo de la fase **Bcast** pasó de 5.611 a 2.474 s por Ethernet y de 88.904 a 40.272 s por Wi-Fi. Los rangos de las cinco repeticiones no se solapan en ninguna red. Con 24 procesos en un nodo no hay diferencia clara (0.649 frente a 0.691 s por Ethernet y 0.666 frente a 0.673 s por Wi-Fi, con rangos solapados): no hay tráfico entre nodos que ahorrar.

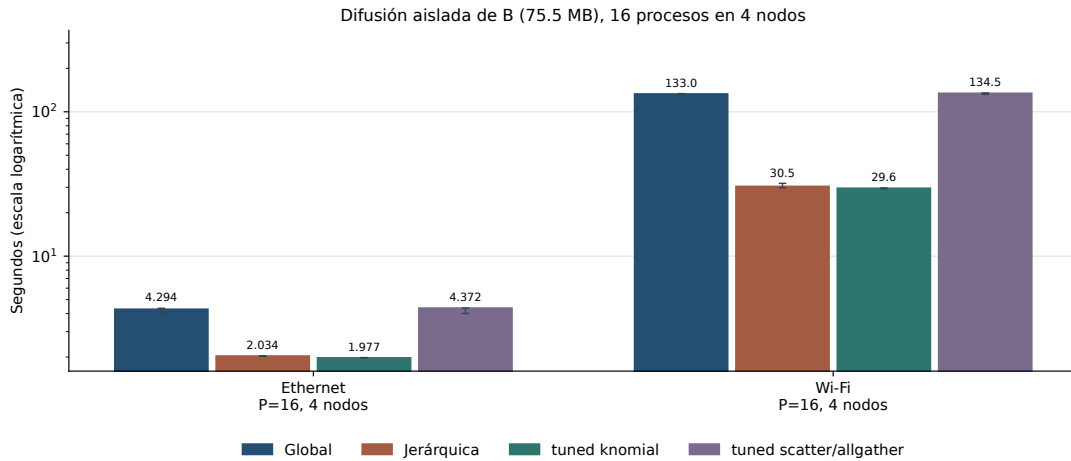


Figura 8: Difusión aislada de B con 16 procesos en cuatro nodos. Barras: mediana de tres repeticiones; líneas: mínimo y máximo. Escala vertical logarítmica.

El microbenchmark aísla el efecto sin el cálculo. La colectiva global y la selección forzada `scatter_allgather` dieron tiempos y TX próximos (1032.5 y 1032.4 MB por Ethernet; 1006.5 MB por Wi-Fi). La jerárquica y la selección forzada `knomial` bajaron el TX a unos 228–238 MB, cerca de las tres copias de 75.5 MB que requieren tres nodos remotos (226.5 MB). **knomial obtuvo una mediana 3 % menor que la versión jerárquica escrita en C** en ambas redes, con rangos separados en estas tres repeticiones. Es una alternativa prometedora sin cambiar el código. Queda por medir esa selección dentro de la multiplicación completa con $P = 96$.

14.4. Partición 2D y tamaño del problema

SUMMA 2D con $P = 64$ no mejoró el tiempo completo: 5.492 s frente a 3.147 s de 1D por Ethernet y 87.314 s frente a 51.728 s por Wi-Fi, con casi el doble de TX (678 frente a 358 MB). El resultado no invalida SUMMA en general; indica que esta implementación, este tamaño y esta topología introducen más pasos y empaquetamiento que los que ahorran.

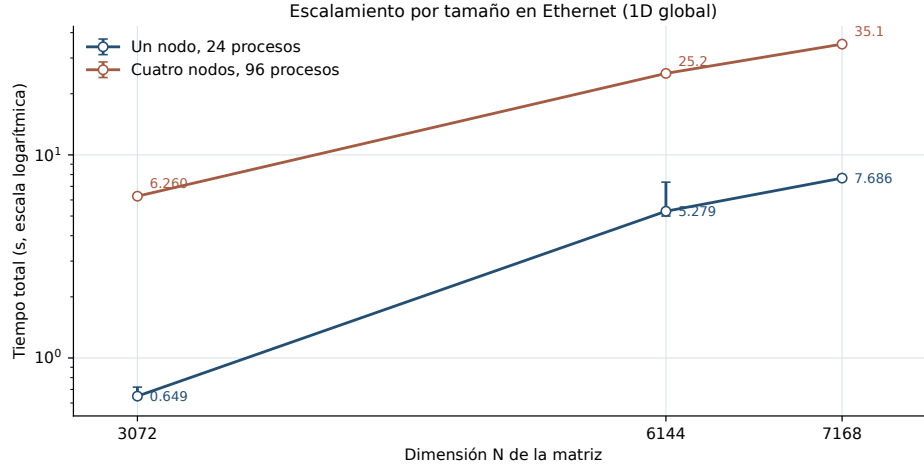


Figura 9: Escalamiento por tamaño en Ethernet con 1D global. Cada punto es la mediana y las barras muestran el rango observado.

Al crecer N , el cociente T_{96}/T_{24} bajó de 9.65 ($N = 3072$) a 4.77 ($N = 6144$) y 4.57 ($N = 7168$). El cálculo crece como N^3 y la comunicación como N^2 , así que la desventaja de los cuatro nodos se reduce, pero no desaparece en los tamaños medidos. Como el binario de la campaña calcula más despacio que el de las baterías, estos cocientes probablemente *subestiman* la ventaja de un solo nodo. No se puede extrapolar un cruce: además, en 1D cada proceso reserva la matriz B completa, de modo que la memoria agregada de los cuatro nodos no permite resolver matrices mayores que las que caben en uno.

14.5. Reparto asimétrico del trapecio

Tabla 2: Trapecio simétrico y asimétrico, tres repeticiones por Ethernet. *Desb.* es el cociente $T_{CalcMax}/T_{CalcMin}$ impreso en cada log.

n, P	Reparto	Rep. 1		Rep. 2		Rep. 3	
		T (s)	Desb.	T (s)	Desb.	T (s)	Desb.
$10^8, 24$	simétrico	0.0052	1.625	0.0072	2.183	0.0076	2.353
	asimétrico	0.0042	1.100	0.0069	1.863	0.0040	1.100
$10^8, 96$	simétrico	0.0020	1.333	0.0057	1.283	0.0013	1.275
	asimétrico	0.0020	1.195	0.0019	1.142	0.0017	1.278
$10^{11}, 24$	simétrico	5.379	1.247	5.183	1.244	4.753	1.286
	asimétrico	4.528	1.272	5.153	1.396	4.940	1.233
$10^{11}, 96$	simétrico	1.004	1.263	1.024	1.290	1.012	1.272
	asimétrico	1.046	1.188	1.026	1.165	1.198	1.360

El reparto asimétrico asigna 1.2658 veces más trapecios a los núcleos P. Con $n = 10^{11}$ y 24

procesos, su mediana fue 4.940 s frente a 5.183 s del simétrico, pero la tercera repetición simétrica (4.753 s) fue más rápida que dos de las asimétricas, y el desbalance asimétrico (1.233–1.396) no quedó por debajo del simétrico (1.244–1.286). Con 96 procesos, el asimétrico fue más lento en las tres repeticiones. Con $n = 10^8$ los tiempos son de milisegundos y el desbalance varía mucho entre corridas. **El reparto ponderado no produce una mejora demostrable con este binario;** el peso 1.2658 se calibró en un microbenchmark exploratorio cuyas salidas no se conservaron.

14.6. Tabla completa de la campaña

La tabla 3 contiene todas las configuraciones válidas. Los tiempos de `bcast` corresponden a `T_Bcast` del microbenchmark; los demás, a `T_Total`.

Tabla 3: Campaña de optimización: medianas de las corridas válidas, con mínimo y máximo. TX suma la transmisión de las interfaces de los nodos participantes.

Red	Prueba	Tamaño	P	Nodos	Variante	Reps.	T med. (s)	Rango (s)	TX (MB)
Ethernet	bcast	3072	16	1	hier	3	0.124	0.117–0.129	0.0
Ethernet	bcast	3072	16	1	world	3	0.116	0.116–0.127	0.0
Ethernet	bcast	3072	16	4	hier	3	2.034	2.030–2.035	238.4
Ethernet	bcast	3072	16	4	tuned-knomial	3	1.977	1.974–1.983	238.4
Ethernet	bcast	3072	16	4	tuned-scatter	3	4.372	4.010–4.374	1032.4
Ethernet	bcast	3072	16	4	world	3	4.294	4.003–4.351	1032.5
Wi-Fi	bcast	3072	16	4	hier	3	30.464	29.791–31.940	228.0
Wi-Fi	bcast	3072	16	4	tuned-knomial	3	29.575	29.354–29.650	228.6
Wi-Fi	bcast	3072	16	4	tuned-scatter	3	134.479	132.263–134.957	1006.5
Wi-Fi	bcast	3072	16	4	world	3	133.043	131.304–133.633	1006.5
Ethernet	matriz	3072	24	1	1d	5	0.649	0.633–0.719	0.0
Ethernet	matriz	3072	24	1	1d-hier	5	0.691	0.677–0.799	0.0
Ethernet	matriz	3072	64	4	1d	3	3.147	3.147–3.148	358.0
Ethernet	matriz	3072	64	4	2d	3	5.492	5.469–5.546	678.1
Ethernet	matriz	3072	96	4	1d	5	6.260	6.233–6.266	755.5
Ethernet	matriz	3072	96	4	1d-hier	5	3.217	3.088–3.288	358.9
Ethernet	matriz	6144	24	1	1d	3	5.279	4.998–7.343	0.6
Ethernet	matriz	6144	96	4	1d	3	25.194	25.065–25.322	3022.4
Ethernet	matriz	7168	24	1	1d	3	7.686	7.561–7.703	0.4
Ethernet	matriz	7168	96	4	1d	3	35.116	35.086–35.130	4113.9
Wi-Fi	matriz	3072	24	1	1d	5	0.666	0.581–0.722	0.0
Wi-Fi	matriz	3072	24	1	1d-hier	5	0.673	0.608–4.769	0.0
Wi-Fi	matriz	3072	64	4	1d	3	51.728	48.975–52.122	343.0
Wi-Fi	matriz	3072	64	4	2d	3	87.314	86.305–87.779	677.7
Wi-Fi	matriz	3072	96	4	1d	5	101.082	100.293–102.991	729.3
Wi-Fi	matriz	3072	96	4	1d-hier	5	51.479	50.817–52.659	342.9
Ethernet	trapecio	10^8	24	1	asimetrico	3	0.004	0.004–0.007	0.0
Ethernet	trapecio	10^8	24	1	simetrico	3	0.007	0.005–0.008	0.0
Ethernet	trapecio	10^8	96	4	asimetrico	3	0.002	0.002–0.002	0.6
Ethernet	trapecio	10^8	96	4	simetrico	3	0.002	0.001–0.006	0.4
Ethernet	trapecio	10^{11}	24	1	asimetrico	3	4.940	4.528–5.153	0.1
Ethernet	trapecio	10^{11}	24	1	simetrico	3	5.183	4.753–5.379	0.0
Ethernet	trapecio	10^{11}	96	4	asimetrico	3	1.046	1.026–1.198	0.5
Ethernet	trapecio	10^{11}	96	4	simetrico	3	1.012	1.004–1.024	0.8

14.7. Resultados que no deben sobreinterpretarse

El hecho de que `1d-hier` sea más rápido no demuestra que la colectiva global anterior enviara exactamente una copia por proceso ni que todo el tráfico local viajara por TCP. Cada proceso sigue reservando su propia copia completa de B ; la variante nueva cambia el patrón de mensajes, no convierte la memoria en una ventana compartida. Para identificar el algoritmo interno de Open MPI en cada corrida habría que registrar el componente colectivo seleccionado.

La suma TX de los nodos y el contador TX+RX del servidor de las baterías anteriores tienen alcances diferentes. Se conservan ambos porque responden preguntas distintas: el primero permite comparar la presión agregada en la campaña nueva y el segundo documenta qué observó una interfaz concreta durante la batería ampliada. Ninguna de las dos cifras equivale al volumen útil exacto de una colectiva.

15. Una corrida frente a repeticiones

Las dos baterías ejecutaron cada configuración una sola vez; la campaña repitió algunas de ellas de tres a cinco veces. Donde coinciden la prueba, el tamaño, la red y el número de procesos, se puede comparar la magnitud de las diferencias entre campañas. Como también cambió la compilación, esas diferencias no estiman por sí solas el error de una corrida única. La tabla usa la variante 1D global de la campaña y, para el trapecio, su modo simétrico.

Tabla 4: Tiempo total (s) de las corridas únicas de cada batería frente a la mediana y el rango de la campaña.

Prueba	Red	P	Corrida única		Campaña		
			Bat. 1	Bat. 2	Mediana	Rango	Reps.
Matrices $N = 3072$	Ethernet	24	0.512	0.547	0.649	0.633–0.719	5
Matrices $N = 3072$	Ethernet	96	6.175	6.148	6.260	6.233–6.266	5
Matrices $N = 3072$	Wi-Fi	24	0.553	0.553	0.666	0.581–0.722	5
Matrices $N = 3072$	Wi-Fi	96	101.042	99.945	101.082	100.293–102.991	5
Matrices $N = 6144$	Ethernet	24	–	3.839	5.279	4.998–7.343	3
Matrices $N = 6144$	Ethernet	96	–	25.037	25.194	25.065–25.322	3
Trapecio $n = 10^{11}$	Ethernet	24	3.870	3.842	5.183	4.753–5.379	3
Trapecio $n = 10^{11}$	Ethernet	96	0.964	0.975	1.012	1.004–1.024	3

15.1. Qué muestra la comparación

- **Con 96 procesos, las corridas únicas de matrices quedaron próximas a las medianas posteriores.** Con $N = 3072$ y $N = 6144$, las baterías quedan a menos de un 2 % de la mediana repetida en las redes medidas. El tiempo está dominado por la comunicación, que dio resultados próximos en los tres momentos.
- **Con 24 procesos, la campaña fue sistemáticamente más lenta:** un 19–27 % con $N = 3072$ y un 37 % con $N = 6144$. Ninguna de las corridas repetidas de $N = 3072$ por Ethernet bajó hasta los tiempos de las baterías.
- **La diferencia está en el cálculo, no en la red.** La tabla 5 separa las fases de las configuraciones con 24 procesos.

Tabla 5: Máximos de cálculo y de distribución con 24 procesos: corridas únicas frente a la mediana de la campaña.

Matriz	Red	T_{Calc} máximo (s)			T_{Dist} máximo (s)		
		Bat. 1	Bat. 2	Campaña	Bat. 1	Bat. 2	Campaña
$N = 3072$	Ethernet	0.304	0.326	0.455	0.198	0.205	0.220
$N = 3072$	Wi-Fi	0.345	0.337	0.450	0.216	0.228	0.214
$N = 6144$	Ethernet	–	2.891	4.447	–	0.875	0.959

T_{Dist} cambia poco (hasta un 11 %), mientras T_{Calc} crece entre 1.3 y 1.5 veces en la campaña (0.304 a 0.455 s con $N = 3072$ por Ethernet; 2.891 a 4.447 s con $N = 6144$). Esto es coherente con la diferencia de compilación: sin `-march=native`, el compilador solo puede vectorizar con el conjunto básico x86-64 (SSE2) y no emite las instrucciones AVX2 y FMA del procesador. No es una prueba aislada, porque también cambió el momento de ejecución; comprobarlo requiere recompilar con los mismos flags y repetir.

El trapecio muestra un patrón similar con 24 procesos (3.84–3.87 s en las baterías frente a 5.18 s de mediana), pero ahí la comparación mezcla además dos programas: las baterías usaron `trapecio.c` y la campaña, `trapecio_asimetrico` en modo simétrico, con la corrección del extremo.

15.2. Consecuencias para la lectura del informe

1. Las conclusiones cualitativas se mantienen: en todas las fuentes, un nodo con 24 procesos es el mejor punto para matrices y la difusión entre nodos domina el tiempo con 96 procesos.
2. Los cocientes T_{96}/T_{24} de la campaña (4.77 y 4.57) no deben mezclarse con los de las baterías (6.52 para $N = 6144$). Con un binario más rápido en el cálculo, la ventaja de un nodo es mayor.
3. Una diferencia pequeña entre dos corridas únicas con 24 procesos, por ejemplo entre redes, no es interpretable: la variación entre momentos supera esas diferencias.

16. Auditoría de la evidencia

Antes de redactar esta versión se volvió a contrastar cada cifra con su registro primario. El script `auditar_evidencia.py` repite las comprobaciones de forma automática y genera la tabla 6; solo lee los CSV, logs, JSON y sellos del repositorio original y no corrige ningún dato.

16.1. Lo que queda verificado

- **Los tiempos de las tablas y figuras coinciden con los logs.** Las 50 filas de la primera batería, las 75 de la segunda y las 118 de la campaña reproducen exactamente la línea `RESULT` de su log, incluidas las fases y, en la campaña, los contadores TX/RX de cada nodo.
- **Los cálculos son consistentes.** Para cada uno de los seis tamaños de matriz, todas las ejecuciones de ambas baterías imprimen la misma suma diagonal y la misma suma ponderada. El error máximo del trapecio respecto de π es 1.6×10^{-10} .

Tabla 6: Resultado de la auditoría automática.

Comprobación	Alcance	Resultado
Primera batería: CSV frente a logs	50 filas (T_Total, P)	sin discrepancias
Segunda batería: CSV consolidado frente a logs	75 filas (4 tiempos)	sin discrepancias
Huellas de matrices en ambas baterías	6 tamaños de N	sin discrepancias
Error del trapecio respecto de π	40 ejecuciones	máximo 1.61e-10
Campaña: CSV frente a logs	118 filas (tiempos y huellas)	sin discrepancias
Campaña: TX/RX frente a contadores del log	118 filas	sin discrepancias
Campaña: sellos SHA256SUMS	122 archivos	sin discrepancias
Copias de evidencia frente a procedencia	152 archivos	sin discrepancias
Originales externos, si están disponibles	152 archivos	distintos: graficar.py, mpi_matrix_2d.c
Compilación registrada en la campaña	entorno.json	mpicc -O3 -std=c11 -Wall -Wextra
Gráficos de bateria_expandida/... ./analisis/	metricas.csv	solo Wi-Fi

- **La evidencia no se alteró.** Los 122 archivos sellados de la campaña y las 152 copias listadas en la procedencia conservan su SHA-256.
- **El informe es reproducible.** Las tablas y las figuras PNG del informe original se regeneraron en una copia aislada y resultaron idénticas byte a byte; los PDF de figuras solo difirieron en la fecha de creación.

16.2. Problemas hallados y cómo se tratan

Gráficos de la batería ampliada que no corresponden a su nombre. En la copia original disponible en esta computadora, dentro de `experimentos_96/bateria_expandida/20260925_003457/analisis/`, `metricas.csv` coincide exactamente con `tiempos_wifi.csv`: los ocho gráficos son de Wi-Fi y no incluyen $N = 6144$. `graficar.py` escribe siempre en `analisis/`, junto al CSV, y los dos CSV comparten carpeta, lo que explica que la ejecución Wi-Fi sobrescribiera la de Ethernet. Este informe no usa esos gráficos; sus figuras de la batería ampliada se generan directamente desde el CSV consolidado.

Procedencia de dos archivos. En los originales disponibles en esta computadora, la copia `src/mpi_matrix_2d.c` del repositorio del informe se modificó después para añadir `1d-hier`, y el hash de `procedencia.json` se actualizó a la versión modificada. El programa que produjo las dos baterías es el original de `experimentos_96/src/`, idéntico al del primer commit `f81f488`. El camino `1d` es equivalente en ambas versiones (mismas barreras y colectivas); la versión nueva solo separa los tiempos de `Scatterv` y `Bcast`. A la inversa, `experimentos_96/graficar.py` se modificó después de copiarse (añade el gráfico `04_speedup_vs_master`); la copia conservada es la versión anterior. Estos originales externos no se necesitan para compilar el informe; la auditoría indica cuántos pudo consultar.

Compilación distinta en la campaña. Las baterías usaron `-O3 -march=native -funroll-loops`; la campaña, `-O3` sin optimizaciones específicas del procesador. Sus efectos se analizan en la sección 15.

Afirmaciones de los borradores que no se sostienen. Los documentos de trabajo del directorio `experimentos_96` contienen tablas numéricas correctas, pero también conclusiones que los registros no respaldan. Este informe no las incorpora:

- `HALLAZGOS_Y_CLARIFICACIONES_OPTIMIZACION_96.md` informa una mejora del trapecio asimétrico de 1.19 veces (−16 %) usando solo la primera repetición; con medianas es 1.05 y los rangos se solapan (tabla 2). Su columna de desbalance (1.045, 1.082, 1.341...) no coincide con ningún log de la campaña. Su tabla de tamaños combina mínimos o máximos de $P = 24$ con medianas de $P = 96$ y la variante jerárquica con la global. También atribuye a la campaña el uso de `-march=native`, que `entorno.json` no registra.
- `INFORME_ANALISIS_EXPANDIDO_96.md` supone que las pruebas actuales ya usaban una difusión jerárquica; la campaña lo contradice (755.5 MB frente a 358.9 MB con $P = 96$). Atribuye 357 MB medidos a las pruebas de 96 procesos, cuando el servidor registró 500.9–516.7 MB y los 357 MB son de otra serie. Ajusta porcentajes de sobrecarga para lograr una “concordancia exacta” con 1112 MB. Explica la lentitud de madrugada con una saturación de RAM, aunque el cálculo ocupó 0.44 s de 338.8 s y 1.74 s de 906 s. Además, presenta como resultados la tabla P/E y el “−68 %” de espera, sin salidas conservadas.

Datos históricos con respaldo parcial. Las series de madrugada y mediodía se conservan como JSON con dos repeticiones por configuración y sin logs completos, salvo `repetir_profiling_cluster.log`, que registra el caso de 906.351 s. Por eso se tratan como antecedentes y no se promedian con las baterías.

17. Conclusiones y límites de lo que podemos defender

17.1. Conclusiones

1. **El tipo de problema determina si conviene distribuirlo.** El trapecio grande obtiene aceleraciones de 68.56 y 68.20 por Ethernet con 96 procesos en la primera y la segunda batería. En matrices, el mejor tiempo fue siempre un nodo con 24 procesos: en las dos baterías, en todos los tamaños hasta $N = 6144$, y en la campaña hasta $N = 7168$.
2. **La comunicación explica la penalización de matrices.** Con 96 procesos y $N = 3072$, el cálculo máximo ronda 0.10 s en ambas redes, mientras el total es 6.175 s por cable y 101.042 s por Wi-Fi. Las fases de distribución y reunión concentran los tiempos grandes.
3. **Reducir las copias entre nodos es la optimización que funcionó.** La difusión jerárquica redujo a la mitad el tiempo y el tráfico con 96 procesos en las dos redes, con rangos separados en cinco repeticiones. La selección `knomial` de Open MPI produjo TX similar y una mediana ligeramente menor en la difusión aislada, sin cambiar el código.
4. **No todas las ideas de optimización se confirmaron.** SUMMA 2D fue más lenta que 1D con $P = 64$, y el reparto asimétrico P/E no mostró una mejora consistente en el trapecio.
5. **La lentitud de madrugada está registrada, pero no se reproduce.** La corrida de 906.351 s usó 8 procesos, dos en cada uno de cuatro nodos, y pasó 889.744 s en el máximo

de **Bcast**; su repetición tardó 322.513 s con el mismo cálculo. Otra serie, con 16 procesos, midió unos 1112 MB en 1D y 423 MB en 2D. No son cifras de una misma ejecución.

6. **Optimizar caché y reducir tráfico son mejoras distintas.** Los bloques aceleran el cálculo local; la partición y la difusión cambian las comunicaciones. Añadir **vader** explicita los transportes, pero no prueba por sí solo la causa del cambio de volumen.
7. **La comparación entre campañas depende del peso del cálculo.** En matrices con 96 procesos, las corridas únicas quedan a menos de un 2 % de las medianas posteriores; con 24 procesos difieren entre un 19 y un 37 % entre momentos y binarios.

17.2. Qué cambió respecto del informe anterior

Ahora sí se usa Ethernet; ya no queda como una propuesta futura. Se incorporó el trapecio, se llegó a 96 procesos y se incluyó un punto explícito con un nodo completo. La segunda batería amplió tamaños, registró TX+RX del servidor y contrastó sus 75 logs con CSV y JSON. La campaña de optimización añadió repeticiones, variantes de difusión, SUMMA 2D y tamaños mayores, con sellos SHA-256. Esta versión revisada añade la auditoría de la evidencia y el análisis de variabilidad.

17.3. Qué haría falta para confirmar las causas pendientes

- Recompilar la campaña con `-O3 -march=native -funroll-loops` y repetir las configuraciones con 24 procesos, para separar el efecto del binario del momento de ejecución.
- Medir la multiplicación completa con $P = 96$ y la selección **knomial**, para confirmar que dos opciones MCA sustituyen a la versión jerárquica escrita a mano.
- Registrar el componente y el algoritmo colectivo elegidos en cada corrida, junto con los bytes por nodo.
- Repetir las baterías con al menos tres corridas por configuración, alternando redes y en orden aleatorio.
- Añadir contadores TCP y de radio para estudiar reintentos, y contadores de memoria para contrastar el modelo de caché.

Estas verificaciones quedan **propuestas**; no se presentan como experimentos ya realizados.

La conclusión más defendible es que el clúster acelera los trabajos con suficiente cálculo independiente, mientras las comunicaciones pueden anular esa ventaja. Reducir las copias de datos entre nodos fue la mejora más grande y reproducible para matrices. Las cifras del informe permiten mostrarlo sin ocultar los resultados desfavorables ni convertir modelos de tráfico en mediciones.

A. Configuración, compilación y comprobación

Estos comandos usan el proyecto existente. Son instrucciones de reproducción; no se ejecutaron nuevas mediciones durante la elaboración del informe.

A.1. Comprobar el software y los nodos

```
1 cd /home/alumno16/experimentos_96
2 mpirun --version
3 mpicc --showme:command
4 lscpu
5
6 for h in 10.7.50.203 10.7.50.201 10.7.50.204; do
7     ssh -F ssh_config "$h" \
8         'hostname; nproc; mpirun --version | head -1'
9 done
10
11 # Diagnostico de conectividad; ping no mide ancho de banda.
12 ping -c 3 10.7.50.203
13 ping -c 3 10.7.134.58
```

A.2. Compilar los dos programas

```
1 cd /home/alumno16/experimentos_96
2 mkdir -p /var/tmp/mpi_trapecio /var/tmp/mpi_bloques
3
4 mpicc -O3 -march=native -funroll-loops \
5     src/trapecio.c -o /var/tmp/mpi_trapecio/trapecio -lm
6
7 mpicc -O3 -march=native -funroll-loops \
8     src/mpi_matrix_2d.c \
9     -o /var/tmp/mpi_bloques/mpi_matrix_2d -lm
```

-O3 activa optimizaciones; -march=native permite instrucciones de la CPU local. El despliegue presupone los equipos compatibles documentados. /var/tmp conserva normalmente los archivos entre reinicios, aunque una política de limpieza puede borrarlos; comprobar su existencia sigue siendo necesario.

A.3. Copiar y verificar los binarios

```
1 for h in 10.7.50.203 10.7.50.201 10.7.50.204; do
2     ssh -F ssh_config "$h" \
3         'mkdir -p /var/tmp/mpi_trapecio /var/tmp/mpi_bloques'
4     scp -F ssh_config /var/tmp/mpi_trapecio/trapecio \
5         "$h:/var/tmp/mpi_trapecio/"
6     scp -F ssh_config /var/tmp/mpi_bloques/mpi_matrix_2d \
7         "$h:/var/tmp/mpi_bloques/"
8 done
9 sha256sum /var/tmp/mpi_trapecio/trapecio \
10    /var/tmp/mpi_bloques/mpi_matrix_2d
11 for h in 10.7.50.203 10.7.50.201 10.7.50.204; do
12     ssh -F ssh_config "$h" \
13         'sha256sum /var/tmp/mpi_trapecio/trapecio /var/tmp/mpi_bloques/mpi_matrix_2d'
14 done
```

B. Comandos de lanzamiento y medición

B.1. Una ejecución, paso a paso

La interfaz de los dos lanzadores es `experimento tamaño procesos`. Se aceptan los procesos 1, 24, 48, 72 y 96.

```

1 cd /home/alumno16/experimentos_96
2
3 # Referencia de un proceso, nodo completo y cuatro nodos.
4 ./lanzar.sh trapezio 100000000000 1
5 ./lanzar.sh trapezio 100000000000 24
6 ./lanzar.sh trapezio 100000000000 96
7
8 # Matrices: misma version 1d tiled 64 en todos los casos.
9 ./lanzar.sh matrices 3072 1
10 ./lanzar.sh matrices 3072 24
11 ./lanzar.sh matrices 3072 96
12 ./lanzar_wifi.sh matrices 3072 96

```

Para ver el comando expandido y la afinidad basta conservar la salida del lanzador. Las líneas `COMANDO` de los logs documentan qué se ejecutó; no basta el nombre de la carpeta.

B.2. Batería completa y CSV

```

1 # Mostrar el plan; no ejecuta los experimentos.
2 ./medir.sh todos --plan
3 ./medir_wifi.sh todos --plan
4
5 # Ejecutar una red por vez, sin otra bateria simultanea.
6 ./medir.sh todos
7 ./medir_wifi.sh todos
8
9 # Tambien se puede elegir solo un experimento.
10 ./medir.sh trapezio
11 ./medir_wifi.sh matrices

```

Cada batería crea un directorio nuevo con fecha y sufijo aleatorio, un log por prueba y `tiempos.csv`. El CSV contiene `experimento,tamano,procesos,T_Total`. Una fila es una ejecución; no se hacen promedios ocultos. Al terminar se invoca `graficar.py`. Si una ejecución falla o no entrega un tiempo válido, el script se detiene.

B.3. Qué hace el script de medición

El script recorre tamaños y procesos, ejecuta el lanzador, extrae `T_Total` de la salida y lo añade al CSV. El cronómetro está en C; `sed` solamente lee su resultado. No hace falta `time` del shell para obtener el tiempo del algoritmo.

B.4. Regenerar gráficos de las corridas actuales

Los siguientes comandos reutilizan datos ya guardados:

```

1 cd /home/alumno16/experimentos_96
2 eth=resultados/20260924_202548_RnIhhM/tiempos.csv
3 wifi=resultados_wifi/20260924_203957_VSNYOP/tiempos.csv
4
5 python3 graficar.py "$eth"
6 python3 graficar.py "$wifi" --red Wi-Fi

```

```

7 python3 comparar_eth_vs_wifi.py "$eth" "$wifi" \
8   --salida comparacion_eth_vs_wifi

```

Se generan tablas de métricas y gráficos de tiempo, speedup y eficiencia. La comparación calcula T_W/T_E . Los gráficos de este informe se regeneran con un script propio que toma copias de los mismos datos y de los históricos, con estilos aptos para impresión en blanco y negro.

B.5. Reconstruir este informe

```

1 cd /home/alumno16/cluster-mpi-96-optimizaciones/revision
2 ./compilar.sh

```

`compilar.sh` ejecuta tres scripts y compila `informe_mejorado.tex`: `generar_datos.py` (tablas y figuras de las baterías y de las series históricas), `generar_optimizacion.py` (tablas y figuras de la campaña) y `auditar_evidencia.py` (auditoría y variabilidad). Los tres leen la evidencia de la carpeta superior del mismo repositorio sin modificarla; otra ubicación se indica con la variable `EVIDENCIA_MPI`. No se llama a `mpirun`, no se cambian las configuraciones de los workers ni se repiten los experimentos.

B.6. Lectura de un resultado real

La línea siguiente se toma del log actual de matrices Wi-Fi, $N = 3072$, $P = 96$. Se muestran solo los campos necesarios para leer las fases:

```

1 N=3072, Procs=96, Algo=1d, Kernel=tiled, BS=64
2 T_Dist=95.931070, T_Calc=0.100977
3 T_Gather=58.381650, T_Total=101.042337

```

`T_Dist` reúne `Scatterv` y `Bcast`; la versión usada en las baterías no registra cada una por separado. La versión de la campaña añade `T_Scatter` y `T_Bcast`. El valor de `T_Gather` puede incluir espera mientras otros procesos aún distribuyen o calculan. Por eso no se suma a los otros máximos para reconstruir el total.

C. Cómo registrar los MB en una próxima ejecución

La primera batería no guarda MB; la segunda registra solo TX+RX del servidor. Para una futura prueba puntual que mida las cuatro interfaces se incluye `medir_trafico.sh`. Lee los contadores de la interfaz elegida en los cuatro nodos y llama al lanzador existente una sola vez. Consulta los workers por Ethernet para reducir el tráfico de control agregado a una medición Wi-Fi; al medir Ethernet esas consultas sí pueden contribuir al contador. Sigue siendo una medida de interfaces, sin aislamiento de tráfico ajeno.

```

1 cd /home/alumno16/cluster-mpi-96-optimizaciones
2
3 # Ejemplo: UNA ejecucion adicional con registro de MB.
4 # No forma parte de los resultados de este informe.
5 ./medir_trafico.sh wifi matrices 3072 96
6
7 # Para Ethernet, cambiar solo el primer argumento:
8 ./medir_trafico.sh ethernet matrices 3072 96

```

El nuevo script deja `trafico.csv`, `antes.tsv`, `despues.tsv` y `ejecucion.log` en una carpeta nueva. Los archivos de antes y después conservan los contadores brutos, además del total calculado. No afirma que sus MB equivalgan solo a la carga útil de MPI ni a los bytes radiados.

C.1. Aislar el efecto de una opción

Para estudiar específicamente `vader` o el algoritmo de `Bcast`, no se debe comparar 16 procesos `ikj` con 96 `tiled` y atribuir toda la diferencia a una opción. Haría falta conservar el mismo tamaño, kernel, número de procesos y asignación. Los comandos de consulta siguientes permiten inventariar los parámetros disponibles en la versión instalada:

```
1 ompi_info --version
2 ompi_info --param btl vader --level 9
3 ompi_info --param coll tuned --level 9
```

Consultar parámetros no demuestra cuál se eligió en una corrida anterior. Habría que registrar también la selección durante la ejecución o forzar explícitamente un algoritmo compatible en un experimento controlado. Es una comprobación pendiente.

D. Scripts actuales conservados

Las copias de esta sección corresponden a los lanzadores y a la medición de la batería analizada. Los archivos de configuración SSH y los demás scripts acompañan al informe en `scripts/`; no se incluyen contraseñas ni claves privadas.

D.1. Lanzador Wi-Fi

El lanzador Ethernet usa el mismo esquema de asignación con las IP de cable. Se incluye el Wi-Fi completo para dejar visibles las opciones que interesan en la comparación histórica.

```
1 #!/usr/bin/env bash
2 # Lanzador por Wi-Fi (interfaz wlp129s0f0, red 10.7.134.0/23)
3 # Uso: ./lanzar_wifi.sh <trapecio/matrices> <tamaño> <1|24|48|72|96>
4 set -euo pipefail
5 export LC_ALL=C
6 unset DISPLAY
7 DIR=$(cd -- "$(dirname -- "$0")" && pwd)
8
9 if [[ $# != 3 ]]; then
10     echo "Uso: $0 <trapecio/matrices> <tamaño> <1|24|48|72|96>" >&2
11     exit 2
12 fi
13 experimento=$1
14 tamano=$2
15 procesos=$3
16 if [[ ! $tamano =~ ^[1-9][0-9]*$ ]]; then
17     echo "El tamaño debe ser un entero positivo." >&2
18     exit 2
19 fi
20
21 # Mapeo de hosts por Wi-Fi (IPs 10.7.134.x)
22 case "$procesos" in
23     1) hosts='10.7.134.117:1' ;;
24     24) hosts='10.7.134.117:24' ;;
25     48) hosts='10.7.134.117:24,10.7.134.58:24' ;;
26     72) hosts='10.7.134.117:24,10.7.134.58:24,10.7.134.59:24' ;;
27     96) hosts='10.7.134.117:24,10.7.134.58:24,10.7.134.59:24,10.7.134.51:24' ;;
28     *) echo "Procesos permitidos: 1, 24, 48, 72, 96" >&2; exit 2 ;;
29 esac
30
31 if (( tamano < procesos )); then
32     echo "El tamaño debe ser al menos igual al número de procesos." >&2
33     exit 2
34 fi
35 case "$experimento" in
36     trapecio) programa=(/var/tmp/mpi_trapecio/trapecio "$tamano") ;;
37     matrices) programa=(/var/tmp/mpi_bloques/mpi_matrix_2d "$tamano" 1d tiled 64) ;;
38     *) echo "Experimento permitido: trapecio o matrices" >&2; exit 2 ;;
```

```

39 esac
40
41 # OpenMPI forzado a la subred Wi-Fi
42 comando=(mpirun -H "$hosts" -np "$procesos"
43     --map-by slot --rank-by slot --bind-to core --report-bindings
44     --mca plm_rsh_agent "ssh -F $DIR/ssh_config_wifi"
45     --mca btl self,vader,tcp
46     --mca btl_tcp_if_include 10.7.134.0/23
47     --mca oob_tcp_if_include 10.7.134.0/23
48     "${programa[@]}")
49
50 printf 'COMANDO (Wi-Fi): '; printf '%q ' "${comando[@]}"; printf '\n'
51 exec "${comando[@]}"

```

D.2. Medición Bash y generación de gráficos

```

1  #!/usr/bin/env bash
2  # Uso: ./medir.sh [todos|trapezio|matrices] [--plan]
3  set -euo pipefail
4  export LC_ALL=C
5  cd -- "$(dirname -- "$0")"
6
7  seleccion=${1:-todos}
8  plan=${2:-}
9  if [[ $seleccion == --plan ]]; then seleccion=todos; plan=--plan; fi
10 case "$seleccion" in
11     todos) experimentos=(trapezio matrices) ;;
12     trapezio|matrices) experimentos=("$seleccion") ;;
13     *) echo 'Uso: ./medir.sh [todos|trapezio|matrices] [--plan]' >&2; exit 2 ;;
14 esac
15 [[ $# -le 2 && ( -z $plan || $plan == --plan ) ]] || exit 2
16
17 if [[ $plan != --plan ]]; then
18     mkdir -p resultados
19     salida=$(mktemp -d "resultados/$(date +%Y%m%d_%H%M%S)_XXXXXX")
20     echo 'experimento,tamano,procesos,T_Total' > "$salida/tiempos.csv"
21     echo "Resultados: $PWD/$salida"
22 fi
23
24 for experimento in "${experimentos[@]}; do
25     if [[ $experimento == trapezio ]]; then
26         tamanos=(100000000 100000000000)
27     else
28         tamanos=(1024 2048 3072)
29     fi
30     for n in "${tamanos[@]}; do
31         for p in 1 24 48 72 96; do
32             echo "./lanzar.sh $experimento $n $p"
33             [[ $plan == --plan ]] && continue
34             log="$salida/${experimento}_${n}_${p}.log"
35             if ! ./lanzar.sh "$experimento" "$n" "$p" > "$log" 2>&1; then
36                 echo "Falló la ejecución. Revisa $log" >&2; exit 1
37             fi
38             # Extrae T_Total de la línea RESULT que imprime el programa C.
39             tiempo=$(sed -nE '/^RESULT_(TRAP|2D):/s/.*T_Total=([0-9]+\.[0-9]+),.*\/1/p' "$log")
40             if [[ ! $tiempo =~ ^[0-9]+\.[0-9]+$ || ! $tiempo =~ [1-9] ]]; then
41                 echo "Tiempo ausente o inválido. Revisa $log" >&2; exit 1
42             fi
43             echo "$experimento,$n,$p,$tiempo" >> "$salida/tiempos.csv"
44             echo "T_Total = $tiempo segundos"
45         done
46     done
47 done
48
49 # Al terminar correctamente, calcula las métricas y genera los gráficos.
50 if [[ $plan != --plan ]]; then
51     python3 graficar.py "$salida/tiempos.csv"
52 fi

```

La variante `medir_wifi.sh` llama a `lanzar_wifi.sh`, guarda en `resultados_wifi/` e identifica

la red al generar los gráficos. Ambos scripts validan que el tiempo exista y sea positivo; **no validan automáticamente los checksums**.

E. Archivos de respaldo y fuentes

El repositorio `cluster-mpi-96-optimizaciones` incluye una copia de la evidencia necesaria para reproducir las tablas y los gráficos; las rutas de la tabla son relativas a la carpeta principal. Esta versión revisada no duplica la evidencia: la lee desde la carpeta superior. `datos/procedencia.json` relaciona cada copia con su ruta original y su hash SHA-256; `datos/validacion.json` resume las comprobaciones. `datos/metricas_informe.csv` conserva las métricas calculadas sin el redondeo de las tablas.

Ruta en el repositorio	Contenido
<code>datos/ethernet/</code>	25 resultados de la primera batería y sus logs por cable.
<code>datos/wifi/</code>	25 resultados de la primera batería y sus logs Wi-Fi.
<code>datos/ampliada/</code>	75 logs y cuatro archivos de resultados de la segunda batería.
<code>datos/procedencia_ampliada.json</code>	Rutas y SHA-256 de los 79 archivos nuevos de evidencia y tres fuentes añadidas.
<code>datos/historicos/resultados_profiling_repeticion.json</code>	Primera serie de madrugada; contiene los 906.351 s con 8 procesos.
<code>datos/historicos/resultados_bloques.json</code>	Caché, escalado local y serie de madrugada con MB y dos repeticiones en el clúster.
<code>datos/historicos/resultados_matrices_eth_vs_wifi.json</code>	Serie del mediodía con tiempos y MB por red.
<code>src/</code>	Fuentes de trapecio, matrices y kernels. <code>mpi_matrix_2d.c</code> es la versión de la campaña (con <code>1d-hier</code>); la usada en las baterías está en <code>experimentos_96/src/</code> y en el commit <code>f81f488</code> .
<code>resultados_optimizacion/</code>	CSV, logs, <code>entorno.json</code> y SHA256SUMS de la campaña de optimización.
<code>fuentes/INFORME_ANALISIS_EXPANDIDO_96.md</code>	Borrador de análisis conservado como fuente, con sus afirmaciones revisadas.
<code>fuentes/exploratorios/</code>	Prototipos de núcleos P/E, sin salidas individuales verificables en la batería.
<code>scripts/</code>	Copias de lanzadores, medición, gráficos y configuración SSH.
<code>generar_datos.py</code>	Lee la evidencia, valida consistencia y genera tablas y seis figuras. Esta versión usa una copia adaptada que solo escribe en su propia carpeta.
<code>medir_trafico.sh</code>	Herramienta opcional para una futura medición de MB.

El contexto actualizado del proyecto orientó la revisión, pero las cifras finales se tomaron de los CSV y logs. Se corrigieron diferencias de tiempo y atribuciones entre series: 1.053217 s para trapecio grande Wi-Fi de la primera batería, 906.351 s en la serie original con 8 procesos, y 1112 MB en la serie posterior con 16 procesos. La primera batería no midió MB; la segunda midió TX+RX del servidor, que no equivale al TX agregado histórico.

Referencias

- [1] Ian Foster. *Designing and Building Parallel Programs*, 1995. Capítulo 2: Designing Parallel Algorithms. <https://www.mcs.anl.gov/~itf/dbpp/text/node14.html>.

- [2] Open MPI. *FAQ: Tuning the run-time characteristics of MPI TCP communications*. Documentación para Open MPI 4.x y anteriores; apartados sobre transportes locales y selección de interfaces. <https://www.open-mpi.org/faq/?category=tcp>.
- [3] Open MPI, versión 4.1.6. *coll_tuned_decision_fixed.c*, función `ompi_coll_tuned_bcast_intra_dec_fixed`. Código fuente oficial. https://github.com/open-mpi/ompi/blob/v4.1.6/ompi/mca/coll/tuned/coll_tuned_decision_fixed.c.
- [4] Registros del laboratorio del 24 de septiembre de 2026. Archivos CSV, JSON, logs y fuentes enumerados en este anexo; copias y hashes incluidos en el paquete del informe.